

IoT4Schools

“Bringing the Internet of Things in school education as a tool to address 21st century challenges”

Budowa ekosystemu SmartHome z wykorzystaniem urządzeń IoT

Przewodnik dla nauczyciela

Autor: George Papaioannou

Afiliacja: ATERMON BV

Licencja: CC BY-NC 4.0 LEGAL CODE, Attribution-NonCommercial 4.0 International



**Co-funded by
the European Union**

Finansowane przez Unię Europejską. Wyrażone poglądy i opinie są jednak poglądami i opiniami wyłącznie autora/autorów i niekoniecznie odzwierciedlają poglądy Unii Europejskiej ani Europejskiej Agencji Wykonawczej ds. Edukacji i Kultury (EACEA). Ani Unia Europejska, ani EACEA nie ponoszą za nie odpowiedzialności.

Spis treści

1 Wprowadzenie do projektu.....	3
1.1 Scenariusz i zakres projektu.....	3
1.2 Cele edukacyjne.....	3
1.3 Ścieżka nauczania – etapy realizacji.....	3
1.4 Wymagania wstępne.....	4
1.5 Sprzęt i oprogramowanie.....	4
1.6 Plan czasowy.....	6
2 Realizacja projektu.....	8
2.1 Poziom 1 – Konfiguracja RPi Pico W i blink.py.....	8
2.1.1 Płytki Raspberry Pi Pico W.....	8
2.1.2 Programowanie – blink.py.....	12
2.1.3 Proces tworzenia obwodu.....	13
2.2 Poziom 1 – Podstawy MicroPythona i wyjaśnienie komponentów.....	14
2.2.1 Podstawy języka MicroPython.....	14
2.2.2 Wyjaśnienie komponentów elektronicznych.....	19
2.3 Poziom 1 – Odczyt wartości z czujników: DHT11, PIR, LDR.....	24
2.3.1 Proces tworzenia obwodów.....	24
2.3.2 Programowanie.....	26
2.4 Poziom 1 – Aktuatory: diody LED & serwomechanizm.....	28
2.4.1 Proces tworzenia obwodu.....	28
2.4.2 Programowanie.....	30
2.5 Poziom 2 – Integracja systemu & Adafruit IO.....	34
2.5.1 Proces budowy obwodu.....	34
2.5.2 Programowanie.....	34
3 Wskazówki i rekomendacje.....	44
4 Referencje.....	44

1 Wprowadzenie do projektu

1.1 Scenariusz i zakres projektu

W tym projekcie uczniowie stworzą system Smart Home korzystając z płytki Raspberry Pi Pico W, różnych czujników i elementów wykonawczych, programowanych w MicroPythonie oraz zintegrowanych z platformą Adafruit IO do monitorowania i sterowania. Projekt ma na celu wprowadzenie uczniów do koncepcji IoT, programowania oraz zarządzania danymi w chmurze, a także rozwijanie ich wiedzy na temat technologii automatyki domowej. Realizacja projektu Smart Home IoT w klasie szkoły średniej obejmuje połączenie zajęć teoretycznych, ćwiczeń praktycznych oraz pracy zespołowej. Projekt pomoże uczniom zrozumieć technologię IoT, programowanie oraz integrację czujników i elementów wykonawczych z mikrokontrolerem, czego efektem będzie w pełni funkcjonalny system inteligentnego domu.

1.2 Cele edukacyjne

- Zapoznanie się z programowaniem w MicroPythonie
- Nauka podłączania elementów elektronicznych i czujników na płytce stykowej
- Tworzenie aplikacji online sterującej wejściami/wyjściami
- Zrozumienie koncepcji Smart Home i urządzeń IoT

1.3 Ścieżka nauczania – etapy realizacji

Projekt składa się z 2 poziomów oraz 8 sesji wymienionych poniżej:

Poziom 1

Sesja 1: Wprowadzenie do koncepcji IoT i zastosowań w obszarze inteligentnych domów oraz prezentacja i dyskusja na temat podstaw IoT i rzeczywistych przykładów inteligentnych domów.

Sesja 2: Omówienie Raspberry Pi Pico W i jego możliwości oraz demonstracja konfiguracji Raspberry Pi Pico W i podstawowego projektu migającej diody LED w MicroPythonie.

Sesja 3: Wprowadzenie do podstaw programowania w MicroPythonie i proste ćwiczenia z kodowania, aby zapoznać studentów ze składnią i poleceniami MicroPythona. Wyjaśnienie komponentów (płytki stykowej, rezystor, kable połączeniowe, siłowniki, czujniki).

Sesja 4: Podłączanie i programowanie czujnika DHT11, czujnika PIR (detekcji ruchu) i czujnika światła (LDR).

Sesja 5: Podłączanie i programowanie diod LED i serwomechanizmu. Pisanie skryptów do sterowania nimi na podstawie sygnałów z czujników.

Poziom 2

Sesja 6: Stworzenie kompletnego układu z całą elektroniką i czujnikami oraz kompletnego programu w MicroPythonie. Konfiguracja konta Adafruit IO i tworzenie kanałów danych. Wysyłanie danych z czujników do Adafruit IO i sterowanie siłownikami za pośrednictwem Adafruit IO.

Sesja 7: Testowanie i debugowanie kompletnego systemu inteligentnego domu. Przygotowanie dokumentacji i prezentacji dla każdej grupy roboczej.

Sesja 8: Studenci prezentują swoje projekty inteligentnych domów.

Nauczyciele powinni pamiętać, że pewne kwestie należy poruszać w trakcie lekcji, sami decydując, kiedy i gdzie je omówić oraz jak bardzo utrudnić je swoim uczniom.

1.4 Wymagania wstępne

Obowiązują następujące wymagania wstępne:

1. Studenci powinni posiadać podstawową wiedzę z zakresu programowania (zmienne, pętle, instrukcje warunkowe i funkcje (w dowolnym języku)). Doświadczenie w MicroPythonie nie jest wymagane.
2. Studenci powinni znać interfejsy programistyczne oraz rozumieć, czym są interpreter, edytor, terminal i powłoka.
3. Studenci powinni posiadać podstawową wiedzę na temat IoT, pojęcia „chmura” oraz istoty inteligentnego domu.
4. Studenci powinni posiadać podstawową wiedzę na temat obwodu elektrycznego, bezpiecznego korzystania z płytki stykowej oraz odczytywania prostych schematów wyprowadzeń.
5. Studenci powinni umieć interpretować proste odczyty czujników (temperatury, natężenia światła).

W odniesieniu do wymagań wstępnych dla nauczyciela/klasy, obowiązują następujące wymagania:

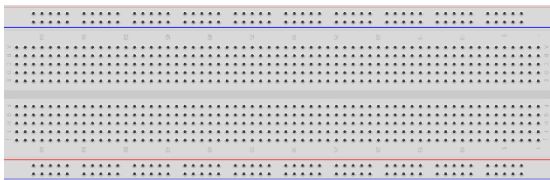
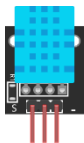
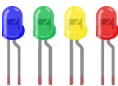
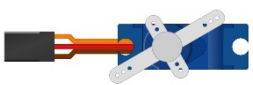
1. Nauczyciele powinni upewnić się, że Thonny jest poprawnie zainstalowany na komputerach/laptopach w klasie.
2. Nauczyciele powinni pomóc uczniom w instalacji oprogramowania MicroPython na RPi Pico W.
3. Nauczyciele powinni upewnić się, że szkolna sieć Wi-Fi umożliwia połączenie RPi Pico W (2,4 GHz, WPA2).
4. Nauczyciele powinni utworzyć konto Adafruit IO dla całej klasy i udostępnić indywidualne kanały każdej grupie uczniów.

Zaleca się również przeprowadzenie na początku instruktażu dotyczącego bezpieczeństwa, w tym maksymalnego natężenia prądu na pinach Pico, oraz aby nigdy nie zasilać urządzenia jednocześnie przez USB i baterie.

1.5 Sprzęt i oprogramowanie

1 x Raspberry Pi Pico W



1 x Płytki stykowa	
1 x DHT11	
1 x czujnik PIR HC-SR501	
1 x fotorezystor LDR	
2 x dioda LED	
1 x serwomechanizm SG-90	
1 x moduł zasilania MB-102	

	
4 x rezystor 220 Omów + 1 x rezystor 10K Omów	
Koszyczek na 6 baterii AA i baterie AA	
Przewody połączeniowe (męsko-męskie i męsko-żeńskie)	

1.6 Plan czasowy

Całkowity czas na budowę i naukę: ~7-8 godzin lekcyjnych (6,5–7,5 godziny)

Zalecane tempo: 2–3 sesje tygodniowo przez ~3–4 tygodnie

Czas przygotowania nauczyciela: ~60–90 minut przed sesją 1; następnie 15–30 minut na sesję

Oto zalecenia dla każdego poziomu i każdej sesji:

Poziom 1

Sesja 1 (45–50 min) — IoT w inteligentnych domach

- 10 min: Wprowadzenie i cele edukacyjne, przykłady inteligentnych urządzeń.
- 15 min: Miniwykład: Komponenty IoT (czujnik → mikrokontroler → chmura → siłownik).
- 15 min: Aktywność zespołowa: Mapowanie „pomieszczeń i czujników” dla scenariusza inteligentnego domu.
- 5–10 min: Krótkie prezentacje scenariuszy uczniów.

Sesja 2 (45–50 min) — Konfiguracja Pico W i pierwsze mignięcie diody LED

- 10 min: Przegląd sprzętu (piny Pico W, zasilanie, płyta stykowa).
- 10 min: Szybki start Thonny + MicroPython.
- 20 min: Tworzenie pliku blink.py w Thonny.
- 5–10 min: Refleksja.

Sesja 3 (45–50 min) — Podstawy MicroPythona i wyjaśnienie komponentów

- 5 min: Podsumowanie.
- 20 min: Kodowanie z przewodnikiem: „Hello IoT4schools!”.
- 20 min: Wyjaśnienie komponentów (płyta stykowa, rezystor, kable połączeniowe, siłowniki, czujniki).

Sesja 4 (45–50 min) — Odczyt czujników: DHT11, PIR, LDR

- 5 min: Zasady działania czujników i bezpieczeństwo pinów.
- 30 min: Podłączenie i odczyt każdego czujnika; wyświetlenie wartości w powłoce Thonny.
- 10–15 min: Rejestrowanie prostych zdarzeń (np. wykrycie ruchu → drukowanie znacznika czasu).

Sesja 5 (45–50 min) — Siłowniki: diody LED i sterowanie serwomechanizmami

- 5 min: Koncepcja PWM dla serwomechanizmów.
- 25 min: Sterowanie kątem serwomechanizmu; wskaźniki wielodiodowe LED.
- 15–20 min: Połączenie z jednym czujnikiem (np. PIR → działanie serwomechanizmu/diody LED).

Poziom 2

Sesja 6 (60–90 min) — Integracja systemu + Adafruit IO

- 10 min: Tworzenie kanałów (temperatura, wilgotność, światło, ruch, servo_cmd, led_cmd).
- 20–30 min: Podłączanie Pico W do Wi-Fi; testowanie publikowania danych z czujników za pomocą feed-ów.
- 20–30 min: Subskrypcja feed; sterowanie diodą LED/serwo z pulpitu nawigacyjnego.

- 10–20 min: Budowanie prostego pulpitu nawigacyjnego Adafruit IO (przełączniki, suwaki, wskaźniki).

Sesja 7 (45–50 min) — Testowanie, debugowanie, dokumentowanie

- 10 min: Ustrukturyzowany plan testów (przypadki dla każdego czujnika/aktuatora + podróż w obie strony po chmurze).
- 25 min: Debugowanie zespołowe z punktami kontrolnymi nauczyciela.
- 10–15 min: Slajdy początkowe/prezentacja jednostronicowa: cele, zdjęcie okablowania, najważniejsze fragmenty kodu, zrzut ekranu pulpitu nawigacyjnego.

Sesja 8 (45–50 min) — Prezentacje

- 5 min: Konfiguracja.
- 30–35 min: Demonstracje zespołowe (po 3–5 min każda).
- 10–15 min: Opinie kolegów i refleksje na temat tematów dotyczących prywatności/zrównoważonego rozwoju.

2 Realizacja projektu

Na poziomie 1 nauczyciele powinni przeprowadzić wprowadzenie do koncepcji i zastosowań Internetu Rzeczy w inteligentnych domach oraz zainicjować dyskusje ze swoimi uczniami na temat podstaw Internetu Rzeczy i rzeczywistych przykładów inteligentnych domów. W tym celu przygotowaliśmy kilka punktów do dyskusji:

- Jak integracja technologii Internetu Rzeczy zmienia tradycyjne systemy automatyki domowej? Jakie są zalety wykorzystania Internetu Rzeczy w automatyce domowej? W jaki sposób systemy Internetu Rzeczy poprawiają wygodę, wydajność i bezpieczeństwo w domach?
- W jaki sposób inteligentne systemy Internetu Rzeczy (IoT) mogą przyczynić się do zrównoważonego rozwoju środowiska i oszczędności? W jaki sposób inteligentne systemy domowe mogą zmniejszyć zużycie energii i obniżyć rachunki za media? Jakie są przykłady urządzeń Internetu Rzeczy, które promują ekologiczne praktyki w domach?
- Jakie są potencjalne zagrożenia związane z prywatnością i bezpieczeństwem danych w inteligentnych systemach domowych opartych na Internecie Rzeczy? Jak można chronić dane przesyłane z urządzeń Internetu Rzeczy przed nieautoryzowanym dostępem? Jakie są najlepsze praktyki w zakresie zapewnienia bezpieczeństwa urządzeń Internetu Rzeczy w inteligentnym domu? Można z nich korzystać na różnych sesjach i poziomach, a wybór czasu i sposobu ich wykorzystania należy do nauczyciela.

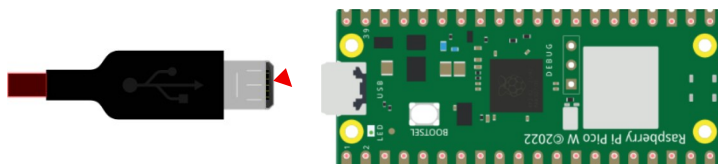
2.1 Poziom 1 – Konfiguracja RPi Pico W i blink.py

2.1.1 Płytki Raspberry Pi Pico W

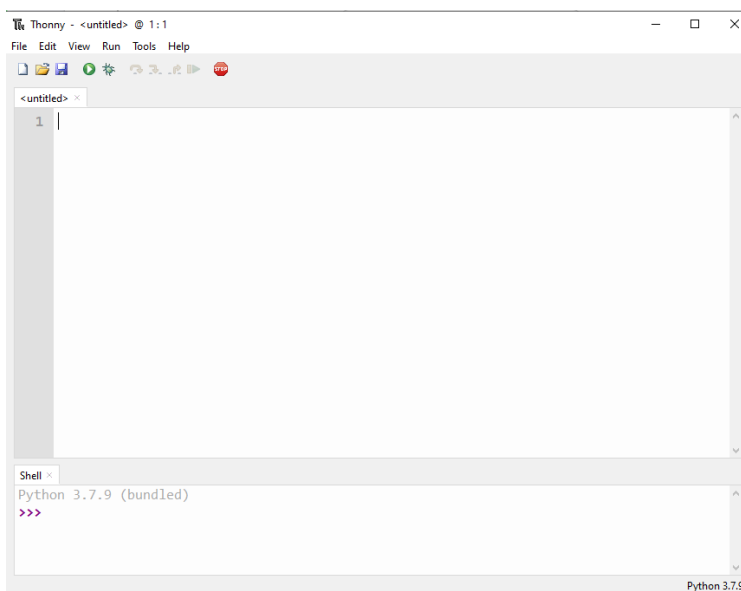
Oto Raspberry Pi Pico W:



Aby podłączyć go do komputera PC/laptopa, potrzebujemy kabla micro-USB. Włóż kabel do portu po lewej stronie płytki.

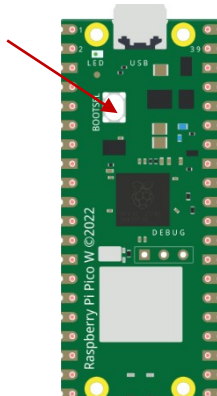


Otwórz edytor Thonny na swoim komputerze:



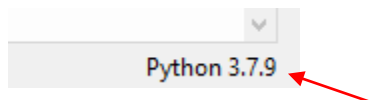
Raspberry Pi Pico można zaprogramować za pomocą wariantu języka Python o nazwie MicroPython. Aby używać MicroPythona na Raspberry Pi Pico, należy najpierw zainstalować jego oprogramowanie.

Krok 1: znajdź przycisk BOOTSEL na Raspberry Pi Pico.

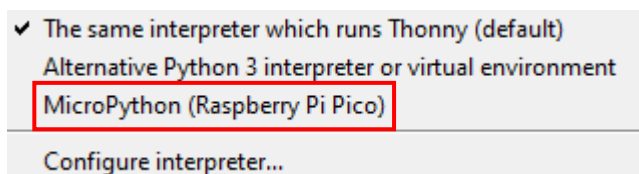


Krok 2: Naciśnij przycisk BOOTSEL i przytrzymaj go, podłączając drugi koniec kabla micro-USB do komputera.

Krok 3: W prawym dolnym rogu Thonny zobaczysz wersję Pythona, której aktualnie używasz.

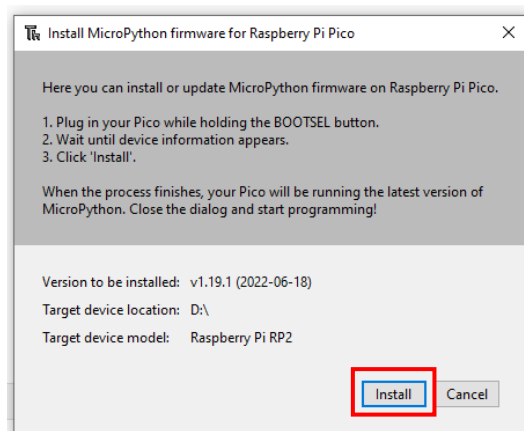


Kliknij wersję Pythona i wybierz MicroPython (Raspberry Pi Pico)

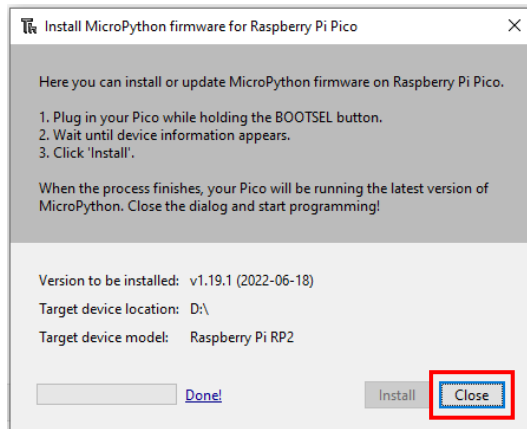


Jeśli nie widzisz tej opcji, upewnij się, że kabel jest prawidłowo podłączony do Pico i/lub komputera.

Krok 4: Pojawi się okno dialogowe z prośbą o zainstalowanie najnowszej wersji oprogramowania układowego w Pico. Kliknij przycisk „Install”, aby skopiować oprogramowanie układowe do Pico.

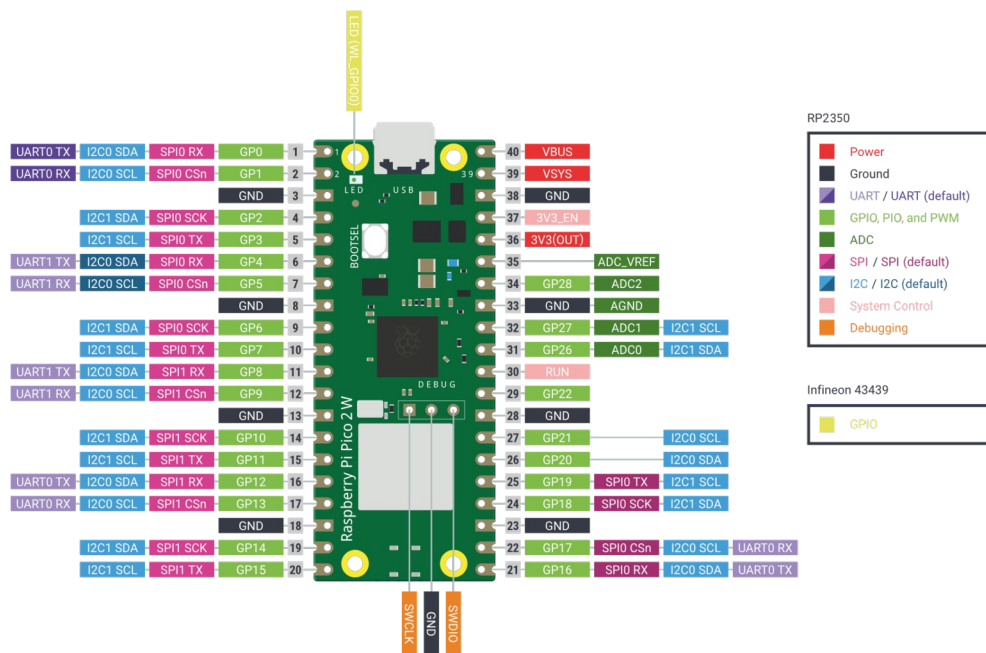


Krok 5: Poczekać na zakończenie instalacji i kliknij **Close**.



Nie musisz powtarzać tego procesu za każdym razem, gdy podłączasz Raspberry Pi Pico do komputera, więc następnym razem po prostu podłącz go do zasilania i gotowe.

W tym projekcie będziesz pracować z pinami ogólnego przeznaczenia (GPIO) Raspberry Pi Pico. Piny te, w większości, obsługują sygnały cyfrowe o wartości 0 lub 1. Jednak trzy piny (GPIO 26, GPIO 27, GPIO 28) są pinami analogowymi. GPIO może pełnić funkcję wejścia (np. odczytując, czy przycisk jest naciśnięty) lub wyjścia (np. włączając i wyłączając diodę LED). Pico ma 40 pinów rozmieszczonych na płytce, numerowanych od lewego górnego rogu. Piny pełniące funkcję GPIO są oznaczone jako GPx (na schematach wyprowadzeń zaznaczone na zielono), gdzie x oznacza liczbę od 0 do 28. Więcej informacji można znaleźć w przewodniku „Techniczny przewodnik po Raspberry Pi Pico i MicroPython” pod adresem: <https://www.iot.fizyka.pw.edu.pl/results/>



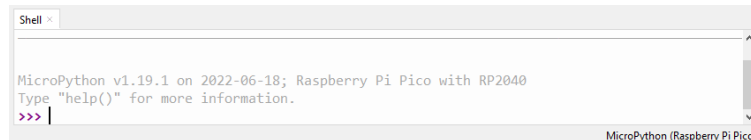
(Image credit: Raspberry Pi) dostępny na <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>

2.1.2 Programowanie – blink.py

Teraz skorzystajmy z środowiska programistycznego Thonny IDE do uruchomienia prostego kodu Pythona, aby zapoznać się z powłoką Thonny i MicroPythonem.

Najpierw upewnij się, że Raspberry Pi Pico jest podłączony do komputera i że wybrałeś interpreter MicroPythona, jak wyjaśniono w poprzedniej sekcji.

Panel powłoki na dole edytora Thonny powinien wyglądać następująco:



```
Shell
MicroPython v1.19.1 on 2022-06-18; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
```

Thonny jest gotowy do komunikacji z Twoim Pico za pomocą środowiska REPL (pętla read-eval-print), które pozwala na pisanie kodu bezpośrednio w powłoce i uzyskiwanie wyników.

Wpisz następujące polecenie:

```
print("Hello!")
```

Następnie naciśnij klawisz Enter, a zobaczysz następujący wynik:



```
Shell
MicroPython v1.19.1 on 2022-06-18; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> print("Hello!")
Hello!
>>>
```

MicroPython pozwala na dodawanie modułów sprzętowych, takich jak moduł *machine*, za pomocą których można programować Pico. W poniższym przykładzie użyjesz modułu *machine* do włączenia wbudowanej diody LED Pico.

Napisz poniższy kod w powłoce Thonny'ego:

```
from machine import Pin
led = Pin("LED", Pin.OUT)
led.on()
```

Naciśnij klawisz Enter, a dioda LED na płytce Pico natychmiast się zaświeci. Aby wyłączyć diodę LED, wpisz poniższy kod:

```
led.off()
```

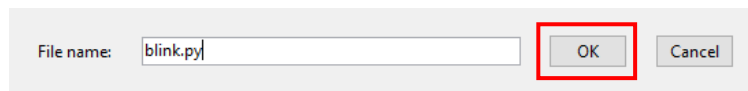
Thonny Shell jest przydatny do testowania szybkich poleceń i upewniania się, że wszystko działa poprawnie. Dłuższe programy należy jednak zapisywać w pliku .py. Za pomocą Thonny możesz zapisywać programy bezpośrednio na Raspberry Pi Pico, a następnie je uruchamiać.

Otwórz Thonny Python i w głównym panelu edytora wpisz następujący kod:

```
from machine import Pin
led = Pin("LED", Pin.OUT)
led.toggle()
```

Teraz zapisz swój program, klikając ikonę „Zapisz” w lewym górnym rogu lub naciskając Ctrl+S na klawiaturze.

Thonny zapyta Cię, gdzie chcesz zapisać swój program. Wybierz **Raspberry Pi Pico**. Zapisz plik jako blink.py i kliknij **OK**. Zawsze musisz dodać rozszerzenie .py, aby Thonny rozpoznał plik jako plik Pythona.



Teraz, za każdym razem, gdy klikniesz ikonę *Play*, powinieneś zobaczyć, jak wbudowana dioda LED włącza się i wyłącza.

Idąc o krok dalej, możesz sprawić, że wbudowana dioda LED będzie migać z określoną częstotliwością.

Napisz poniższy kod i zapisz program pod tą samą nazwą, co powyżej.

```
from machine import Pin
from time import sleep
led = Pin("LED", Pin.OUT)
while True:
    led.toggle()
    sleep(1)
```

Po uruchomieniu programu, wbudowana dioda LED będzie migać co sekundę, aż do momentu zatrzymania programu. Aby zatrzymać program, kliknij ikonę *STOP* lub naciśnij Ctrl+C na klawiaturze.

W kolejnych sesjach nauczysz się, jak dodawać i sterować innymi urządzeniami elektronicznymi i czujnikami oraz tworzyć programy, które mogą się ze sobą komunikować.

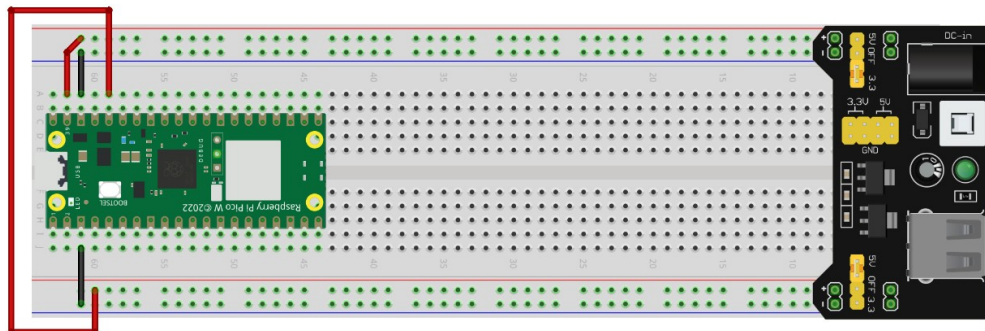
2.1.3 Proces tworzenia obwodu

Przed przystąpieniem do dalszych prac należy wprowadzić kilka modyfikacji na płytce prototypowej (breadboard). Konieczne będzie dodanie modułu Pico do płytki prototypowej wraz z modułem zasilania. Powinny one być prawidłowo podłączone, aby zapewnić przepływ prądu elektrycznego od góry i od dołu płytki prototypowej, co ułatwi podłączanie siłowników i czujników oraz zasilanie ich odpowiednim napięciem.

Potrzebne będą:

- Raspberry Pi Pico W
- Płytki prototypowa
- 1 x moduł zasilania MB-102
- 1 x koszyczek z 6-ma bateriami AA
- 4 x przewody połączeniowe typu „męski-męski”

Postępuj zgodnie ze schematem, aby podłączyć elementy:

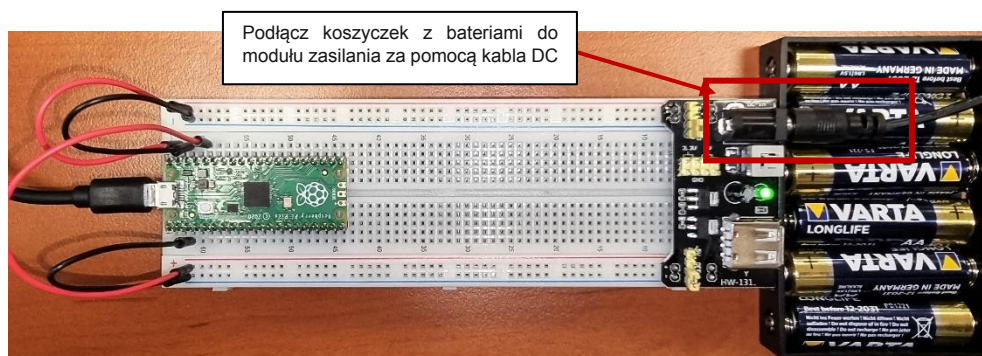


fritzing

Zwróć uwagę na następujące kwestie:

- ta konfiguracja będzie używana w dalszej części projektu.
- połączenia górne: VSYS 5 V ((+) czerwony) i GND ((-) czarny)
- połączenia dolne: 3 V3 ((+) czerwony) i GND ((-) czarny)

Zasadniczo, napięcie na górnej szynie wynosi 5 V, a na dolnej 3 V. Niektóre czujniki wymagają napięcia 5 V do działania, podczas gdy akulatory i prostsze elementy elektroniczne wymagają 3 V. Zostanie to wyjaśnione bardziej szczegółowo w kolejnych rozdziałach. Upewnij się, że połączenia są prawidłowe. Poniższy rysunek pokazuje, jak powinna wyglądać płyta stykowa po podłączeniu modułu zasilania i 6 baterii AA:



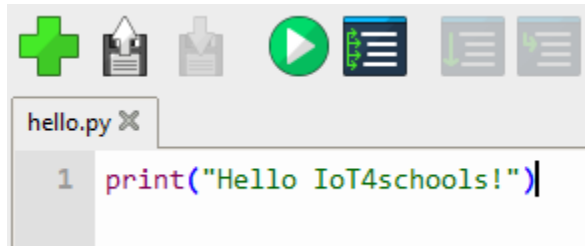
2.2 Poziom 1 – Podstawy MicroPythona i wyjaśnienie komponentów

2.2.1 Podstawy języka MicroPython

Funkcja print()

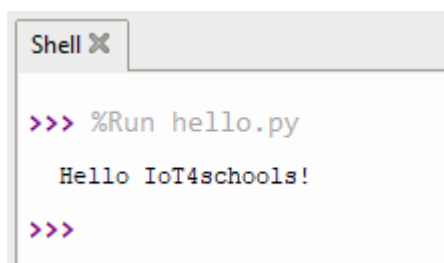
Nauczysz się, jak wyświetlić prostą wiadomość w Thonny za pomocą języka Python.

1. Przejdź do edytora Thonny, wpisz poniższy kod i naciśnij ikonę odtwarzania. Thonny poprosi Cię najpierw o zapisanie programu. Zapisz go pod nazwą hello.py.



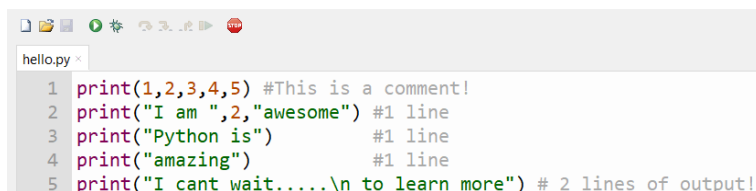
```
hello.py X
1 print("Hello IoT4schools!")
```

2. Sprawdź okno powłoki.



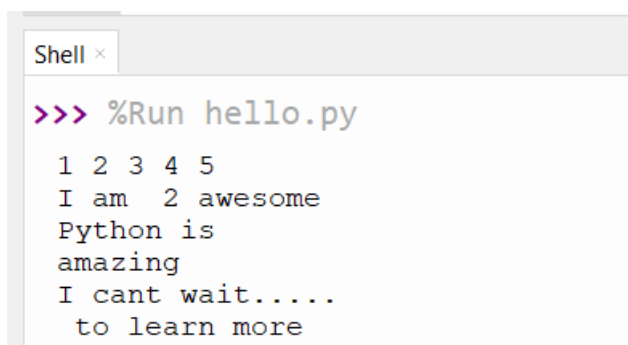
```
Shell X
>>> %Run hello.py
Hello IoT4schools!
>>>
```

Funkcja *print* to wbudowana funkcja Pythona, która pozwala nam wyświetlać tekst w powłoce. Może również przyjmować parametry. Utwórz nowy program i skopiuj poniższy kod, a następnie naciśnij przycisk „Odtwórz” i zobacz, jak tekst pojawi się w powłoce.



```
hello.py X
1 print(1,2,3,4,5) #This is a comment!
2 print("I am ",2,"awesome") #1 line
3 print("Python is") #1 line
4 print("amazing") #1 line
5 print("I cant wait.....\n to learn more") # 2 lines of output!
```

Jak widać w oknie powłoki, każda funkcja *print* wyświetla tekst w osobnym wierszu. Jeśli jednak użyjesz znaku „\n” (nowej linii), możesz zmienić wiersz w tym samym poleceniu drukowania.



```
Shell X
>>> %Run hello.py
1 2 3 4 5
I am 2 awesome
Python is
amazing
I cant wait.....
to learn more
```

Ćwiczenie

Użyj funkcji *print*, aby wyświetlić w 3 oddzielnych wierszach tekst „Żegnaj, ludzie SmartHome!”, używając tylko jednego polecenia *print*.

MicroPython to odmiana Pythona o innej składni, szczególnie w przypadku użycia go z mikrokontrolerami do tworzenia obwodów z aktuatorów, czujników i innych elementów elektronicznych. Poniżej znajduje się zbiorcze podsumowanie większości poleceń i funkcji używanych w tym celu.

Wyjście cyfrowe		
Wywołanie klasy Pin	<code>from machine import Pin</code>	
Inicjalizacja obiektu wyjścia cyfrowego	<code>led = Pin(pin_value, Pin.OUT)</code>	pin_value od 0 do 40
Ustawienie wyjścia cyfrowego w stanie wysokim (3,3 V)	<code>led.value(1)</code>	ON
Ustawienie wyjścia cyfrowego w stanie niskim (0V)	<code>led.value(0)</code>	OFF

Wejście cyfrowe		
Wywołanie klasy Pin	<code>from machine import Pin</code>	
Inicjalizacja obiektu wejścia cyfrowego	<code>button = Pin(pin_value, Pin.IN)</code>	pin_value od 0 do 40
	<code>button = Pin(pin_value, Pin.IN, Pin.PULL_UP)</code>	Aktywacja rezystorów podciągających PULL_UP
	<code>button = Pin(pin_value, Pin.IN, Pin.PULL_DOWN)</code>	Aktywacja rezystorów podciągających PULL_DOWN
Odczyt wartości wejściowej	<code>value = button.value(1)</code>	Wartość zwracana może wynosić 0, jeśli pin ma napięcie 0 V lub 1, jeśli pin ma napięcie 3,3 V

Pulse Width Modulation - PWM		
Wywołanie klasy PWM	<code>from machine import PWM</code>	
Inicjalizacja pinu PWM	<code>led = PWM(Pin(pin_value), frequency)</code>	pin_value od 0 do 40 frequency w Hz, od 0 do 78125
Ustawienie wypełnienia sygnału	<code>led.duty(duty_cycle)</code>	duty_cycle od 0 do 1023

Wejście analogowe		
Wywołanie klasy ADC	<code>from machine import ADC</code>	
Inicjalizacja obiektu wejściowego analogowego	<code>pot = ADC(Pin(pin_value))</code>	pin_value może być: GPIO26, GPIO27 and GPIO28
Deklaracja, przy jakim napięciu wejście da maksymalną wartość (w	<code>pot.atten(ADC.ATTN_11DB)</code>	ADC.ATTN_0DB: napięcie pełnego zakresu: 1,2 V

ESP32 zwykle 3,3 V)		ADC.ATTN_2_5DB: napięcie pełnego zakresu: 1,5 V ADC.ATTN_6DB: napięcie pełnego zakresu: 2,0 V ADC.ATTN_11DB: napięcie pełnego zakresu: 3,3 V
Deklaracja zakresu wartości wejściowych (domyślnie 12 bitów)	pot.width(ADC.WIDTH_10BIT)	ADC.WIDTH_9BIT: zakres od 0 do 511 ADC.WIDTH_10BIT: zakres od 0 do 1023 ADC.WIDTH_11BIT: zakres od 0 do 2047 ADC.WIDTH_12BIT: zakres od 0 do 4095
Odczyt wejściowy	value = pot.read1()	wartość jest liczbą całkowitą od 0 do maksymalnej wartości zakresu określonego przez instrukcję ADC.WIDTH_#BIT (patrz poprzednio)

Biblioteka time		
Wywołanie klasy sleep	<code>from time import sleep</code>	
Użycie funkcji sleep	<code>sleep(sec)</code>	sec to liczba sekund o ile opóźni się wykonanie programu
Wywołanie klasy time	<code>import time</code>	
Użycie funkcji time	<code>current_time = time()</code>	Zmienna current_time przyjmie wartość liczbową równą liczbie sekund od ostatniego resetu płytki.

Struktura if	
<pre> if <expr1>: <statement1> elif <expr2>: <statement2> elif <expr3>: <statement3> (...) else: </pre>	<expr#>: warunek sterujący, który musi zwrócić wartość True lub False <statement#>: zestaw poleceń do wykonania, gdy spełniony jest sąsiedni warunek <expr#> (np. zestaw <statement2> jest wykonywany, gdy spełniony jest <expr2>) <statementn>: zestaw instrukcji wykonywanych, gdy żaden z warunków <expr#> nie jest spełniony

<statementn>	
--------------	--

Pętla while	
<pre>while <expr>: <statement(s)></pre>	<expr>: warunek sterujący, który musi zwrócić wartość Prawda lub Fałsz <statement(s)>: zestaw poleceń, które mają zostać wykonane, o ile spełniony jest warunek <expr>

Pętla for	
<pre>for <var> in <iterable>: <statement(s)></pre>	<iterable>: zbiór obiektów, na przykład lista zawierająca liczby, znaki alfanumeryczne itp. <var>: zmienna, do której przypisywana jest wartość kolejnego elementu w zbiorze <iterable>. <statement(s)>: zestaw instrukcji wykonywanych w każdej iteracji
<pre>for <var> in range(<start>, <end>, <step>): <statement(s)></pre>	range(<start>, <end>, <step>): funkcja zwracająca sekwencję liczb od <start> do <end>-1, z różnicą <step> między dwiema kolejnymi liczbami (parametry <start> i <step> są opcjonalne). <var>: zmienna, do której przypisywana jest wartość kolejnego elementu sekwencji wygenerowanej przez range. <instrukcja(i)>: zestaw instrukcji wykonywanych w każdej iteracji

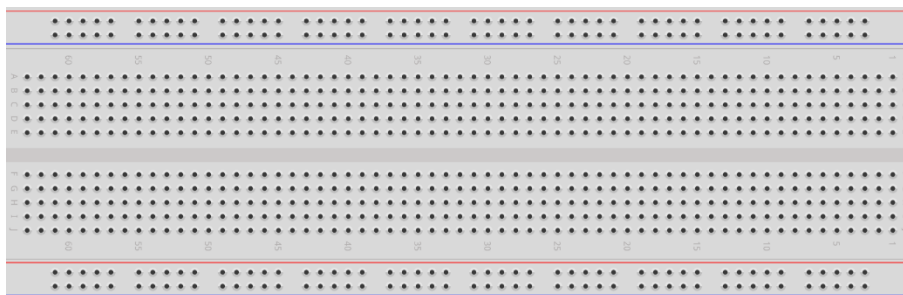
Pozostałe polecenia		
DHT11	<code>import dht</code>	Dodanie biblioteki DHT
	<code>sensor = dht.DHT11(Pin(pin_number))</code>	Inicjalizacja zmiennej czujnika z powiązanym numerem pinu.
	<code>sensor.measure()</code>	Wywołanie pomiaru przez sensor
	<code>temp = sensor.temperature()</code>	Zapisanie obecnej wartości temperatury do zmiennej temp

	<code>hum = sensor.humidity()</code>	Zapisanie obecnej wartości wilgotności do zmiennej <code>hum</code>
OLED DISPLAY	<code>from machine import I2C</code>	Dodanie biblioteki I2C
	<code>import ssd1306</code>	Importowanie biblioteki <code>ssd1306</code>
	<code>i2c = I2C(-1, scl=Pin(1), sda=Pin(0))</code>	Inicjalizacja zmiennej <code>i2c</code> na pinach SCL i SDA Pico
	<code>oled_width = 128</code> <code>oled_height = 64</code> <code>oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)</code>	Inicjalizacja wyświetlacza
	<code>oled.text('Hello, World 1!', 0, 0)</code> <code>oled.text('Hello, World 2!', 0, 10)</code> <code>oled.text('Hello, World 3!', 0, 20)</code>	Przechowywanie wiadomości w buforze ekranowym
	<code>oled.show()</code>	Wyświetlanie wiadomości (niezbędne do wyświetlania wiadomości zapisanych w buforze ekranu)
	<code>display.pixel(3, 4, 1)</code>	Ustaw piksel znajdujący się w pozycji (x,y) na ekranie, przy czym x=3 i y=4, na stan 1 (tj. wyświetlacz)

Porada: Możesz wyjaśnić wszystkie te kwestie swoim uczniom w sposób, który uznasz za odpowiedni, biorąc pod uwagę ich wiedzę i poziom trudności lekcji.

2.2.2 Wyjaśnienie komponentów elektronicznych

Czym jest płytki stykowa?



Płytki stykowa to plastikowa płytka z drobnymi otworami, które umożliwiają łatwe wkładanie elementów elektronicznych (tranzystorów, rezystorów, układów scalonych itp.) w celu prototypowania (budowania i testowania) układu elektronicznego. Wnętrze płytki składa się z rzędów maleńkich metalowych klipsów, które mocują wyprowadzenia do połączenia.

Większość płytek stykowych ma rzędy cyfr, liter oraz znaków plus i minus. Etykiety mają na celu ułatwienie lokalizacji otworów na płycie stykowej, co ułatwia konstruowanie obwodów zgodnie z instrukcjami.

Długie paski po obu stronach płytki stykowej są zazwyczaj oznaczone odpowiednio czerwonymi i niebieskimi lub czerwonymi i czarnymi znakami plus (+) i minus (-). Te rzędy nazywane są szynami lub magistralami i zazwyczaj służą do zasilania obwodu po podłączeniu do źródła zasilania (akumulatora lub zewnętrznego źródła zasilania).

Dodatnia „szyna” jest oznaczona na czerwono, ma znak plus (+) i zapewnia zasilanie.

Ujemna „szyna” jest oznaczona na niebiesko lub czarno, ma znak minus (-) i zapewnia uziemienie.

Zalety korzystania z płytki stykowej:

- Ułatwia szybkie sprawdzanie prostych i złożonych obwodów oraz łatwą weryfikację obwodów na ich początkowym etapie.
- Łatwy montaż.
- Elastyczność.
- Brak wiercenia otworów.
- Nie wymaga lutowania.
- Łatwe debugowanie obwodów i programów.

Czym jest rezystor?



Rezystor to niewielki element oporowy. Użycie go w obwodzie zmniejsza prąd o precyzyjną wartość. Aby określić rezystancję rezystora, należy posłużyć się wzorem kolorowych pasków.

Color	1st Band	2nd Band	3rd Band (5-Band Only)	Multiplier (3rd or 4th Band)	Tolerance (Last Band)
Black	0	0	0	1	
Brown	1	1	1	10	± 1%
Red	2	2	2	100	± 2%
Orange	3	3	3	1000	
Yellow	4	4	4	10000	
Green	5	5	5	100000	± 0.5%
Blue	6	6	6	1000000	± 0.25%
Violet	7	7	7	10000000	± 0.1%
Grey	8	8	8		± 0.05%
White	9	9	9		
Gold				0.1	± 5%
Silver				0.01	± 10%
None					± 1%

(Image credit: Future Owns) dostępne na <https://www.tomshardware.com/how-to/resistor-color-codes>

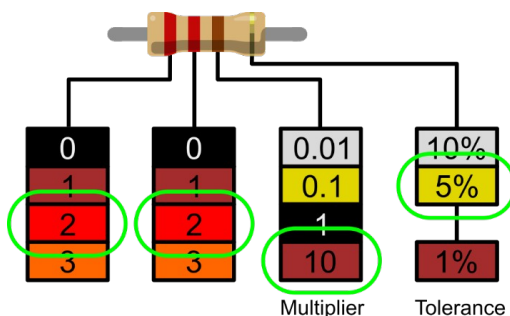
Popularne kody kolorów rezystorów i ich zastosowanie:

Typ rezystora	Kod barwny 4-paskowy	Kod barwny 5-paskowy	Typowe zastosow.
220 Omów	Czerwony- Czerwony- Brązowy- Złoty	Czerwony- Czerwony- Czarny- Czarny-Złoty	Redukcja natężenia przepływającego przez diodę LED
1K Omów (1Kilo omów)	Brązowy- Czarny- Czerwony- Złoty	Brązowy- Czarny- Czarny- Brązowy- Złoty	Dzielnik napięcia

Rezystory nie mają polaryzacji, więc można je stosować w obwodzie w dowolnej orientacji. Aby jednak zidentyfikować prawidłowe wartości kodów kolorów rezystorów, musimy zrozumieć kolorowe paski na rezystorze.

W typowym czteropasmowym rezystorze hobbystycznym grupa składa się z trzech kolorów. Są to pierwsza, druga cyfra znacząca i mnożnik. Ostatni pasek to tolerancja rezystora, czyli margines błędu. Dla większości hobbystów tolerancja 5% (złota) jest idealna i powszechna.

Rezystor 220 Omów (4-paskowy)

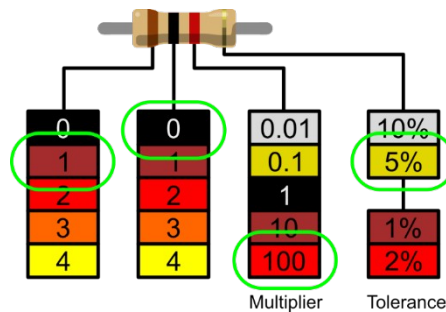


(Image credit: Future) dostępne na <https://www.tomshardware.com/how-to/resistor-color-codes>

1. Pierwsza cyfra znacząca jest czerwona, a korzystając z dekodera, możemy stwierdzić, że czerwony ma wartość 2.
2. Druga cyfra znacząca jest również czerwona, co daje nam 22.
3. Mnożnik jest brązowy, a oznacza on wartość 10. Jeśli pomnożymy 22 przez 10, otrzymamy 220.
4. Ostatni pasek, tolerancja, jest złota. Złoto ma 5%, co oznacza, że możemy zaakceptować rezystancję z 5% marginesem błędu.

Dla użytkowników wymagających większej precyzji dostępne są również rezystory pięciopasmowe z trzecią cyfrą znaczącą. Dodatkowa cyfra zapewnia przejrzystość, co może być niezbędne w obwodach wrażliwych na rezystancję, na przykład w urządzeniach naukowych i inżynierskich.

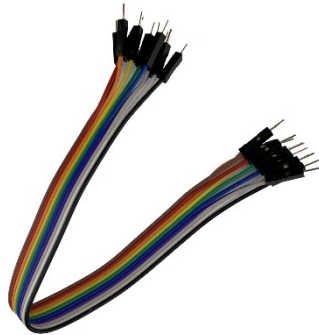
Rezystor 1K Omów (4-paskowy)



(Image credit: Tom's Hardware) dostępne na <https://www.tomshardware.com/how-to/resistor-color-codes>

1. Pierwsza linia jest brązowa, a używając dekodera, możemy odczytać, że oznacza ona wartość 1.
2. Druga linia jest czarna, co daje nam 10.
3. Mnożnik jest czerwony, a dekodowanie wynosi 100. Jeśli pomnożymy 10 przez 100, otrzymamy 1000.
4. Ostatnia linia, tolerancja, jest złota. Złoto ma 5%, co oznacza, że możemy zaakceptować opór z 5% marginesem błędu.

Czym są przewody połączeniowe?

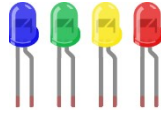


(Image credit: SparkFunEducation) dostępne na <https://blog.sparkfuneducation.com/what-is-jumper-wire>

Kable/przewody połączeniowe to po prostu przewody z pinami połączeniowymi na obu końcach, co pozwala na połączenie dwóch punktów bez lutowania. Przewody połączeniowe są zazwyczaj używane z płytkami stykowymi i innymi narzędziami do prototypowania, aby ułatwić modyfikację obwodu w razie potrzeby. Różnorodność kolorów przewodów może być zaletą w rozróżnianiu typów połączeń, takich jak uziemienie lub zasilanie.

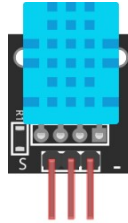
Przewody połączeniowe występują zazwyczaj w trzech wersjach: męsko-męskiej, męsko-żeńskej i żeńsko-żeńskej. Różnica między nimi tkwi w końcówce przewodu. Męskie końcówki mają wystający pin i można je podłączać do różnych urządzeń, podczas gdy żeńskie końcówki nie mają wystającego pinu lecz otwór i służą do podłączania urządzeń. Przewody rozruchowe męsko-męskie są najpopularniejsze. Do łączenia dwóch portów na płytce stykowej najczęściej wymagany jest przewód męsko-męski.

Czym jest dioda LED?



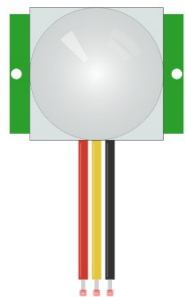
Dioda LED (dioda elektroluminescencyjna) to niewielki półprzewodnik, który świeci, gdy prąd płynie w kierunku przewodzenia. W przeciwieństwie do żarówek, jest spolaryzowana (ma anodę + i katodę –) i wymaga rezystora ograniczającego prąd, aby zapobiec uszkodzeniu; napięcie większości małych diod LED jest w granicach ok. 1,8–3,3 V i toleruje prąd w zakresie 5–20 mA. W elektronice DIY diody LED są podstawowym wskaźnikiem wizualnym – pokazującym zasilanie, stan czujników lub status programu – i mogą być sterowane z pinów GPIO mikrokontrolera, aby migać, ściemniać z PWM lub wyświetlać kolory w wersjach RGB. Są tanie, wydajne i proste w podłączeniu, dzięki czemu idealnie nadają się do szybkiego przekazywania informacji zwrotnych w projektach.

Czym jest sensor DHT11?



Czujnik DHT11 to kompaktowy, cyfrowy czujnik temperatury i wilgotności, szeroko stosowany do odczytów w stacjach pogodowych, automatyce domowej i monitoringu środowiska. Czujnik DHT11 wykorzystuje połączenie pojemnościowego czujnika wilgotności i termistora do pomiaru temperatury i wilgotności w otoczeniu. Podczas pracy czujnik DHT11 generuje sygnały cyfrowe zawierające informacje o temperaturze i wilgotności. Sygnał wyjściowy można połączyć z mikrokontrolerami w celu dalszego przetwarzania. Czujnik DHT11 został zaprojektowany z myślą o rozsądnej dokładności pomiaru temperatury i wilgotności. Należy pamiętać, że, jak w przypadku każdego czujnika, na jego dokładność mogą wpływać takie czynniki, jak kalibracja, warunki środowiskowe i tolerancje produkcyjne.

Czym jest czujnik PIR (HC-SR501)?



Pasywny czujnik podczerwieni (PIR) to czujnik elektroniczny mierzący promieniowanie podczerwone emitowane przez obiekty w jego polu widzenia. Gdy obiekt lub osoba przechodzi przed tym czujnikiem, który może być umieszczony na ścianie lub wewnątrz obiektu, może on wykryć wahania temperatury w podczerwieni. Czujnik przetwarza powstałą zmianę w przychodzącym promieniowaniu podczerwonym na zmianę napięcia wyjściowego. To uruchamia detekcję i umożliwia czujnikowi wysłanie sygnału do systemu łączącego, informując go o zmianie. Dzięki temu czujnik jest idealnym wyzwalaczem dla mechanizmów,

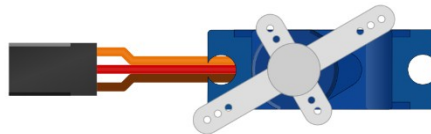
które wolą działać automatycznie po zbliżeniu się, na przykład drzwi wejściowe lub stacje dezynfekcji rąk. Za pomocą wbudowanego potencjometru można określić kilka ustawień. Na przykład, jak długo światło powinno pozostać włączone po wykryciu osoby lub że światło powinno włączać się tylko przy określonej jasności otoczenia (np. w nocy).

Czym jest fotorezystor LDR?



Fotorezystor (LDR) to rezystor zmienny, którego rezystancja zmienia się wraz ze światłem – jasne światło powoduje spadek rezystancji, a ciemność jej wzrost. Nie jest on spolaryzowany i zazwyczaj odczytuje się go poprzez utworzenie dzielnika napięcia z rezystora stałego i pomiar napięcia dzielnika na pinie ADC mikrokontrolera. W projektach DIY jest on wykorzystywany do wykrywania światła otoczenia, automatycznych lampek nocnych, regulacji jasności ekranu/diody LED, alarmów wyzwalanych światłem oraz robotów podążających za linią.

Czym jest serwomechanizm (SG-90)?



Serwomechanizm odbiera sygnał sterujący, który reprezentuje żadaną pozycję lub prędkość, i wykorzystuje układ sprzężenia zwrotnego, zazwyczaj enkoder, do precyzyjnego sterowania i dostosowywania pozycji lub prędkości do sygnału wejściowego. Sterownik silnika porównuje rzeczywistą pozycję lub prędkość silnika z sygnałem wejściowym i w przypadku rozbieżności generuje sygnał korygujący, aby odpowiednio dostosować pozycję lub prędkość silnika. Dzięki temu serwomechanizmy są niezwykle precyzyjne w zadaniach pozycjonowania. Dzięki temu idealnie nadają się do takich zadań, jak tworzenie wjazdów, na przykład szlabanów lub bram garażowych. Są wysoce funkcjonalne, łatwe w obsłudze i można je programować do konkretnych potrzeb.

Modulacja szerokości impulsu (PWM) to technika modulująca współczynnik wypełnienia sygnału cyfrowego w celu sterowania ilością energii dostarczanej do urządzenia lub kodowania informacji w samym sygnale. Zmieniając szerokość impulsu „włączonego” w jednym cyklu, można precyzyjnie kontrolować średnią moc wyjściową lub przysyłać dane, co czyni PWM wysoce skuteczną i wydajną metodą sterowania lub modulacji mocy elektrycznej, między innymi, w zastosowaniach.

2.3 Poziom 1 – Odczyt wartości z czujników: DHT11, PIR, LDR

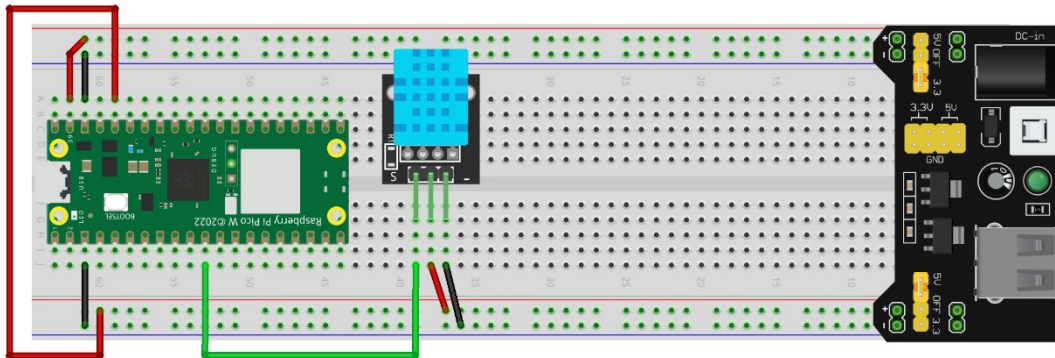
2.3.1 Proces tworzenia obwodów

Czujnik DHT11

Będziesz potrzebować:

- 1 x czujnik DHT11
- 3 x przewody połączeniowe typu „męsko-męskie”

Schemat połączenia:



fritzing

- VCC (czerwony kabel) jest podłączony do szyny 3v3 (+)
- GND (czarny kabel) jest podłączony do szyny GND (-)
- S (zielony kabel) jest podłączony do pinu GPIO8

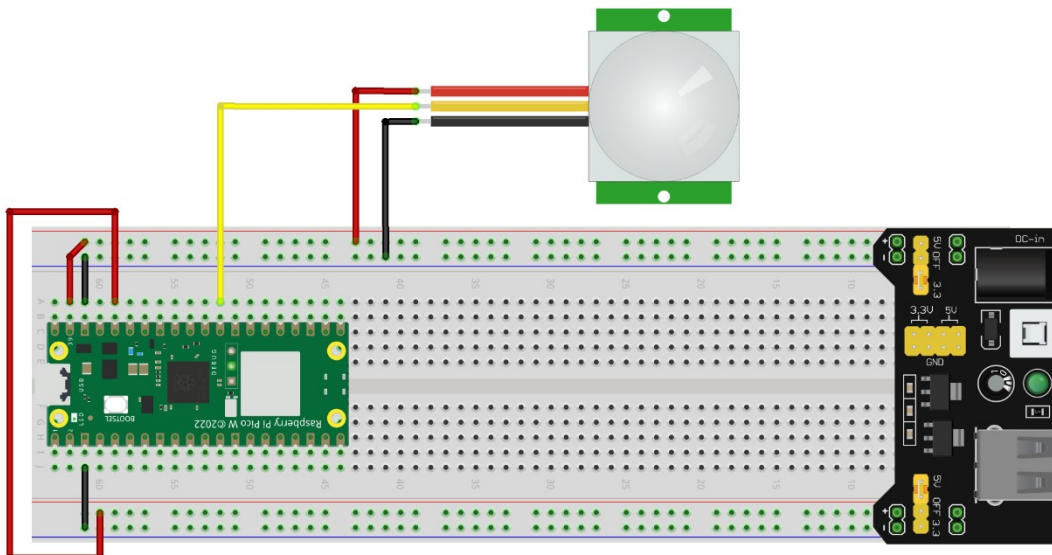
Zauważ, że w tym ćwiczeniu używamy czujnika DHT11 w module KY-015, który ma trzy nóżki.

Czujnik PIR HC-SR501

Będziesz potrzebować:

- 1 x czujnik ruchu PIR
- 3 x przewody połączeniowe typu „męsko-męski”

Schemat połączenia:



fritzing

- VCC (czerwony kabel) jest podłączony do szyny 5 V (+)
- GND (czarny kabel) jest podłączony do szyny GND (-)

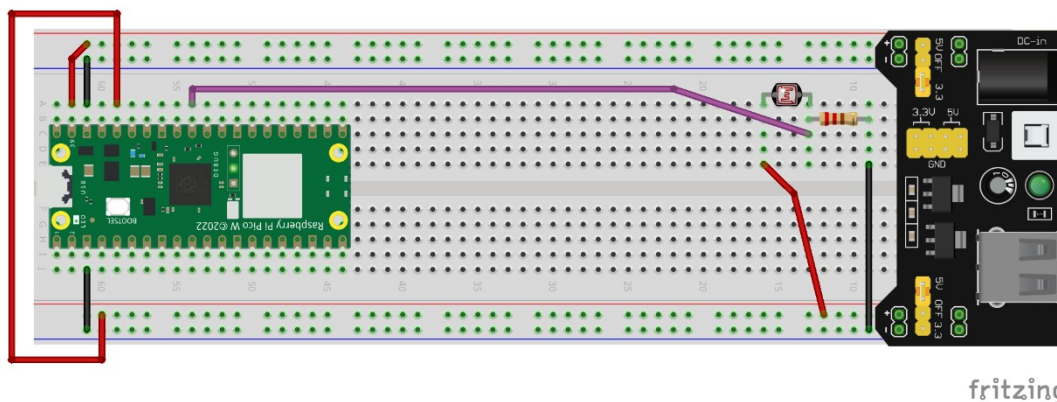
– OUT (żółty kabel) jest podłączony do pinu GPIO22

Fotorezystor LDR

Będziesz potrzebować:

- 1 fotorezystor LDR
- 1 rezystor 10 kΩ
- 3 przewody typu męsko-męskie

Schemat podłączenia:



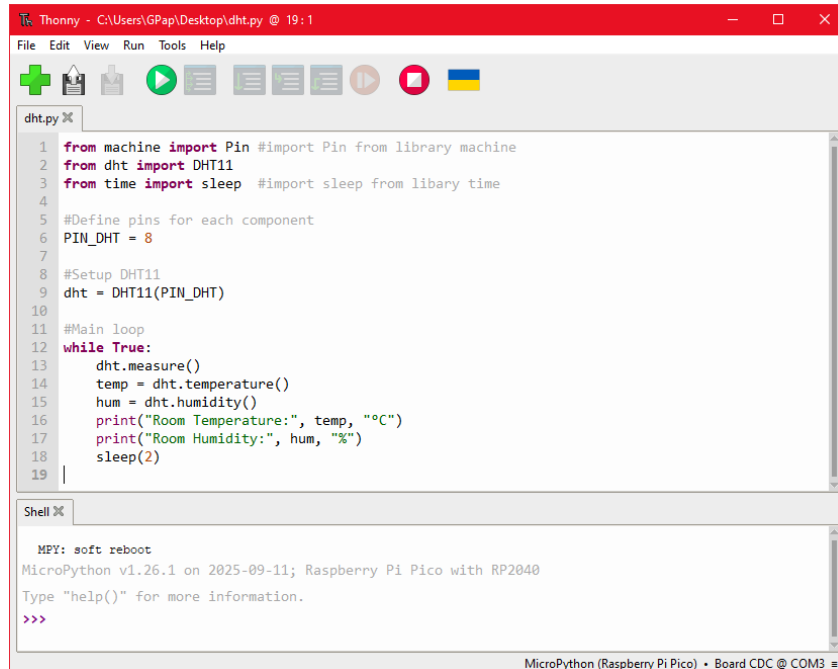
- lewa strona LDR jest podłączona do szyny 3 V ((+) czerwony)
- prawa strona LDR jest podłączona do rezystora 220 Ohm i do pinu GPIO26 ADC0 (fioletowy kabel)
- prawa strona rezystora jest podłączona do szyny GND ((-) czarny)

2.3.2 Programowanie

DHT11

Ten program w MicroPython-ie odczytuje dane z czujnika temperatury i wilgotności DHT11 za pomocą Raspberry Pi Pico i wyświetla wartości w nieskończoność.

- Importuje Pin, DHT11 i sleep, ustawia PIN_DHT = 8 (gdyż pin danych DHT11 był podłączony do GP8) i tworzy obiekt czujnika.
- Wewnątrz, while True:, wywołuje dht.measure(), aby odczytać dane, a następnie pobiera wartości temperature() i moisture() i wyświetla wiersze takie jak Room Temperature: 24 °C i Room Humidity: 48%.
- Używając sleep(2), respektujemy maksymalną częstotliwość aktualizacji DHT11 wynoszącą ~1 Hz.



```

1 from machine import Pin #import Pin from library machine
2 from dht import DHT11
3 from time import sleep #import sleep from library time
4
5 #Define pins for each component
6 PIN_DHT = 8
7
8 #Setup DHT11
9 dht = DHT11(PIN_DHT)
10
11 #Main loop
12 while True:
13     dht.measure()
14     temp = dht.temperature()
15     hum = dht.humidity()
16     print("Room Temperature:", temp, "°C")
17     print("Room Humidity:", hum, "%")
18     sleep(2)
19

```

```

MPY: soft reboot
MicroPython v1.26.1 on 2025-09-11; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>

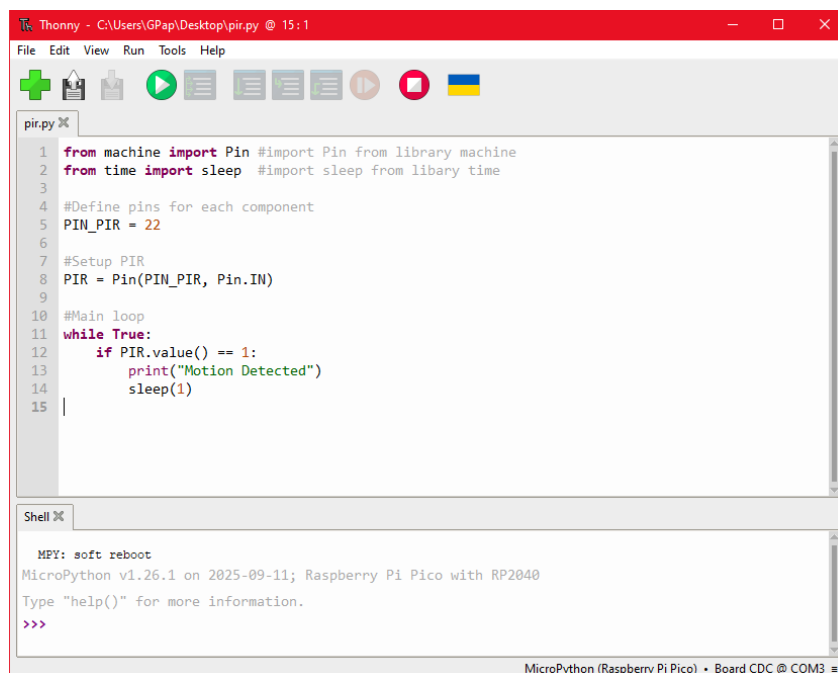
```

MicroPython (Raspberry Pi Pico) • Board CDC @ COM3

Czujnik ruchu PIR HC-SR501

Ten skrypt w MicroPython-ie odczytuje dane z czujnika ruchu PIR za pomocą Raspberry Pi Pico.

- Importuje Pin i sleep oraz ustawia GP22 (PIN_PIR = 22) jako wejście cyfrowe.
- W pętli nieskończonej sprawdza pin: jeśli wejście jest w stanie wysokim (1) – który większość modułów PIR wysyła po wykryciu ruchu – wyświetla komunikat „Motion Detected”.
- sleep(1) spowalnia pętlę, aby uniknąć spamowania konsoli.



```

1 from machine import Pin #import Pin from library machine
2 from time import sleep #import sleep from library time
3
4 #Define pins for each component
5 PIN_PIR = 22
6
7 #Setup PIR
8 PIR = Pin(PIN_PIR, Pin.IN)
9
10 #Main loop
11 while True:
12     if PIR.value() == 1:
13         print("Motion Detected")
14         sleep(1)
15

```

```

MPY: soft reboot
MicroPython v1.26.1 on 2025-09-11; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>

```

MicroPython (Raspberry Pi Pico) • Board CDC @ COM3

Fotorezystor LDR:

Ten skrypt w MicroPython-e odczytuje wartość z fotorezystora (LDR) przez przetwornik analogowo-cyfrowy w Raspberry Pi Pico i wyświetla odczyt raz na sekundę.

- PIN_LDR = 26 wybiera GP26/ADC0.
- LDR = ADC(Pin(PIN_LDR)) ustawia ten pin jako wejście ADC.
- W pętli funkcja LDR.read_u16() zwraca 16-bitową wartość (0–65535) proporcjonalną do napięcia z dzielnika napięcia LDR (jasne światło → niższa wartość).
- sleep(1) przesuwaa odczyty o jedną sekundę.



```

1 from machine import Pin, ADC #import Pin and ADC from library machine
2 from time import sleep #import sleep from library time
3
4 #Define pins for each component
5 PIN_LDR = 26
6
7 #Setup LDR
8 LDR = ADC(Pin(PIN_LDR))
9
10 #Main loop
11 while True:
12     print(LDR.read_u16())
13     sleep(1)
14

```

MPY: soft reboot
MicroPython v1.26.1 on 2025-09-11; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>

MicroPython (Raspberry Pi Pico) • Board CDC @ COM3

2.4 Poziom 1 – Aktuatory: diody LED & serwomechanizm

2.4.1 Proces tworzenia obwodu

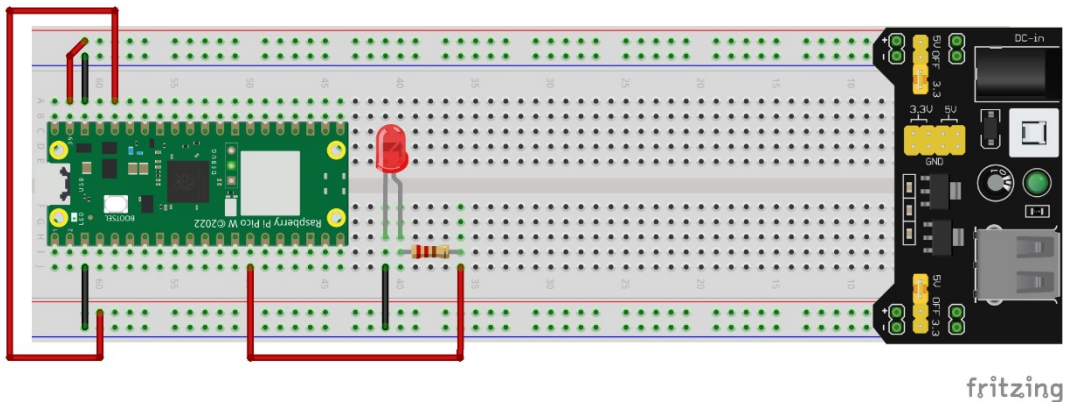
Diody LED

Będziesz potrzebować:

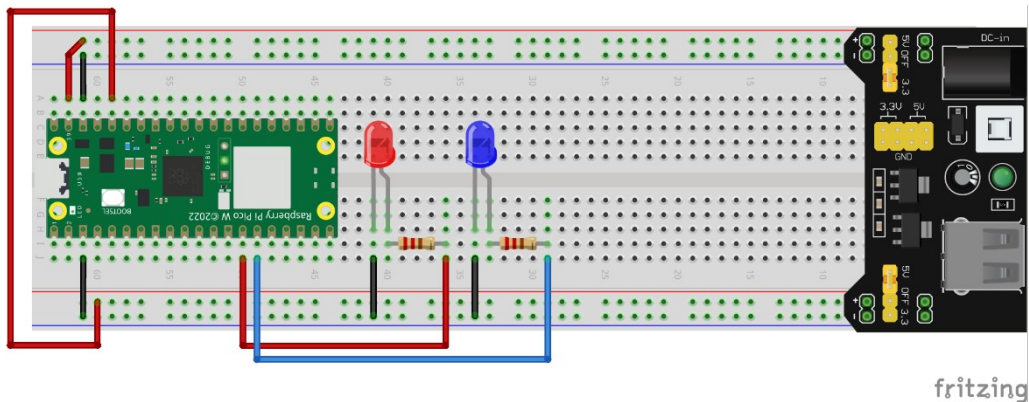
- 2 diody LED
- 2 rezystory 220 omów
- 8 przewodów połączeniowych typu „męsko-męski”

Schemat okablowania:

- Dodanie jednej diody LED:



- Dodawanie wielu diod LED:



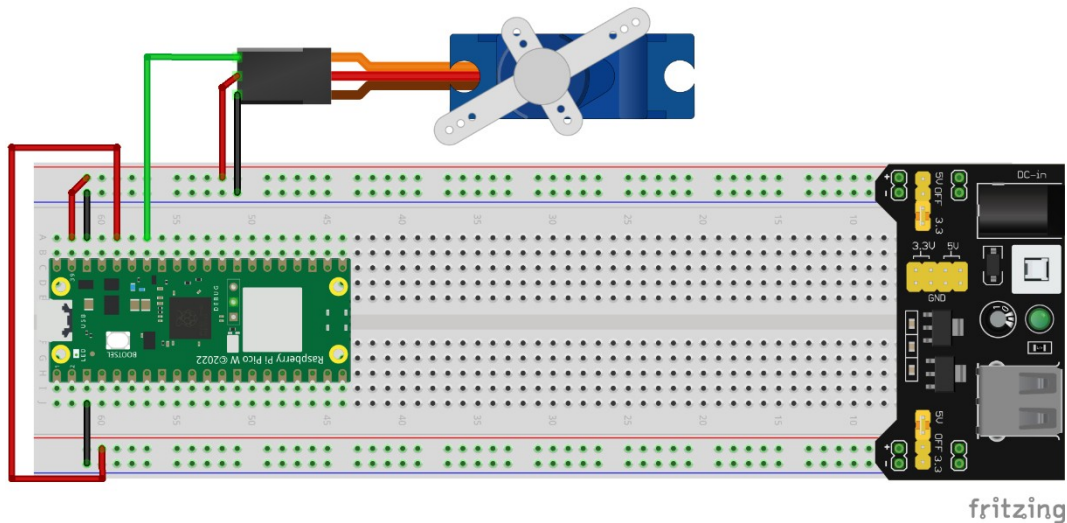
- dla każdej diody LED podłącz dłuższy koniec (+) diody LED do rezystora 220 omów
- dla każdej diody LED podłącz krótszy koniec (-) diody LED do szyny GND ((-) czarny)
- dla czerwonej diody LED podłącz rezystor do GPIO10 (czerwony przewód)
- dla niebieskiej diody LED podłącz rezystor do GPIO11 (niebieski przewód)

Serwomechanizm SG-90

Będziesz potrzebować:

- 1 serwomechanizm SG-90
- 2 przewody połączeniowe typu „męsko-męski”

Schemat podłączenia:



- czerwony kabel jest podłączony do szyny 5V (+)
- czarno-brązowy kabel jest podłączony do szyny GND (-)
- zielony kabel jest podłączony do pinu GPIO28 ADC2

2.4.2 Programowanie

Diody LED

Ten skrypt w MicroPython-e spowoduje miganie zewnętrznej diody LED podłączonej do GP10 na Raspberry Pi Pico. Ustawia GP10 jako wyjście cyfrowe (Pin(LED_RED, Pin.OUT)), wyłącza je jednokrotnie (ledred.value(0)), a następnie wchodzi w nieskończoną pętlę, w której zmienia stan pinu co sekundę (ledred.toggle(); sleep(1)), powodując naprzemienne włączanie i wyłączenie diody LED co sekundę.

```

Thonny - C:\Users\GPap\Desktop\led.py @ 17:1
File Edit View Run Tools Help
+ [Icons] [Run] [Stop] [Python] [Help]

led.py
1 from machine import Pin #import Pin from library machine
2 from time import sleep #import sleep from library time
3
4 #Define pins for each component
5 LED_RED = 10
6
7 #Setup LED lights
8 ledred = Pin(LED_RED, Pin.OUT)
9
10 #Initial value set to off
11 ledred.value(0)
12
13 #Main loop
14 while True:
15     ledred.toggle()
16     sleep(1)
17

Shell
MicroPython v1.26.1 on 2025-09-11; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
MicroPython v1.26.1 on 2025-09-11; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
MicroPython (Raspberry Pi Pico) • Board CDC @ COM3

```


Ten skrypt MicroPython miga czterema diodami LED na Raspberry Pi Pico – czerwoną (GP10), niebieską (GP11), żółtą (GP12) i zieloną (GP13). Konfiguruje każdy pin jako wyjście cyfrowe, wyłącza je jednokrotnie (wartość (0)), a następnie wchodzi w nieskończoną pętlę, w której kolejno przełącza wszystkie cztery diody LED i zatrzymuje się na 1 sekundę. Rezultat: co sekundę wszystkie diody LED zmieniają stan jednocześnie (wyłączona→włączona→wyłączona...).

```

leds.py ✕
1 from machine import Pin #import Pin from library machine
2 from time import sleep #import sleep from library time
3
4 #Define pins for each component
5 LED_RED = 10
6 LED_BLUE = 11
7
8 #Setup LED lights
9 ledred = Pin(LED_RED, Pin.OUT)
10 ledblue = Pin(LED_BLUE, Pin.OUT)
11
12 #Initial value set to off
13 ledred.value(0)
14 ledblue.value(0)
15
16 #Main loop
17 while True:
18     ledred.toggle()
19     ledblue.toggle()
20     sleep(1)
21

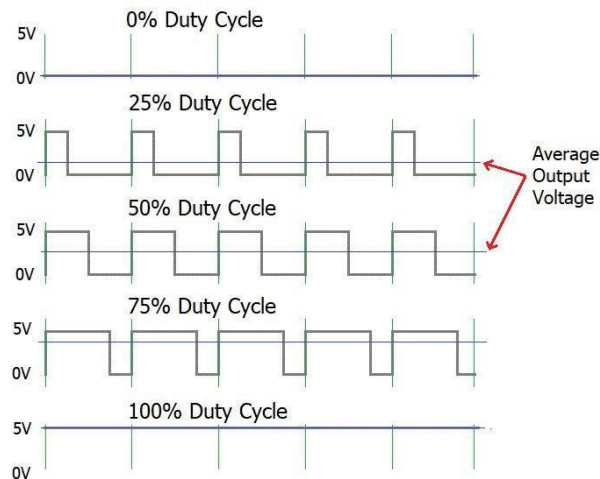
```

Serwomechanizm SG-90

Zanim zaczniesz łączyć elementy i programować serwomechanizm, ważne jest zrozumienie zasady modulacji szerokości impulsu (PWM), która pozwala kontrolować kąt nachylenia serwomechanizmu. Większość mikrokontrolerów nie korzysta z wyjść analogowych, lecz „symuluje” je w specjalny sposób, a konkretnie za pomocą modulacji sygnału.

PWM (modulacja szerokości impulsu) to sposób na wygenerowanie precyzyjnego impulsu „włącz–wyłącz” z pinu cyfrowego. Wybiera się częstotliwość (w przypadku serwomechanizmów takich jak SG90, ~50 Hz → okres 20 ms) i zmienia się szerokość impulsu w każdym okresie. W przypadku serwomechanizmów typu SG90, długość impulsu HIGH koduje kąt docelowy: około 1,0 ms ≈ 0°, 1,5 ms ≈ 90°, 2,0 ms ≈ 180° (wartości różnią się nieznacznie w zależności od serwomechanizmu).

PWM jest potrzebny, ponieważ serwo nie odczytuje stałego napięcia ani poziomu analogowego. Nasłuchuje tych impulsów i porusza wałkiem, aż wewnętrzna pętla sterowania osiągnie zadaną szerokość. Raspberry Pi Pico W może precyzyjnie generować ten czas za pomocą sprzętowego PWM, więc MicroPython po prostu ustawia częstotliwość i dostosowuje wypełnienie (które jest odwzorowywane na szerokość impulsu). Dokładne, stabilne impulsy oznaczają stabilną pozycję serwa; nierównomierne taktowanie oznacza niestabilny ruch.

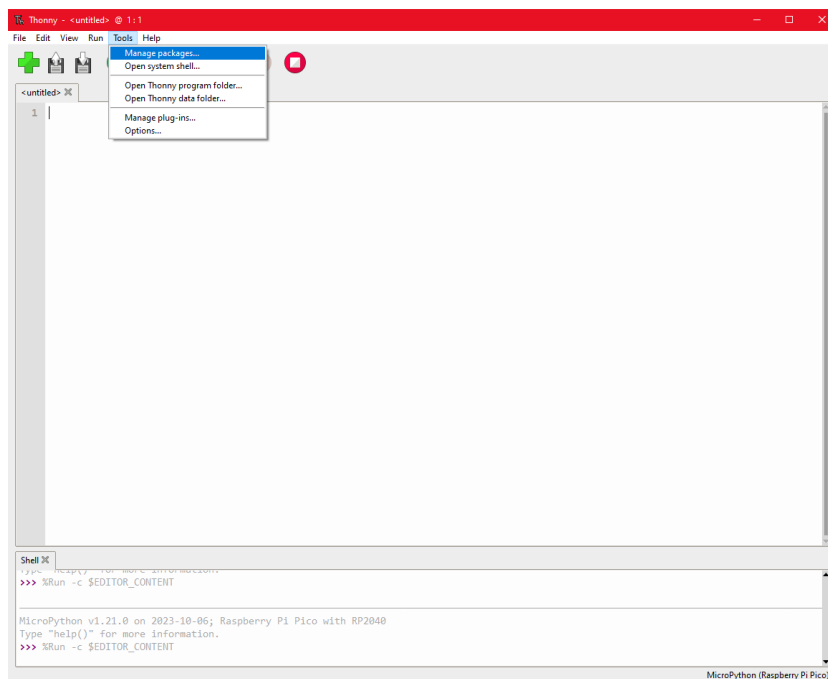


Lam, Jessica & Chen, Audrey & Bennett, Andrew & Triantafyllou, Michael. (2023). *Propeller Characterization Testing of a Blue Robotics T200 Thruster*. 1-8. 10.1109/OCEANSLimerick52467.2023.10244513.

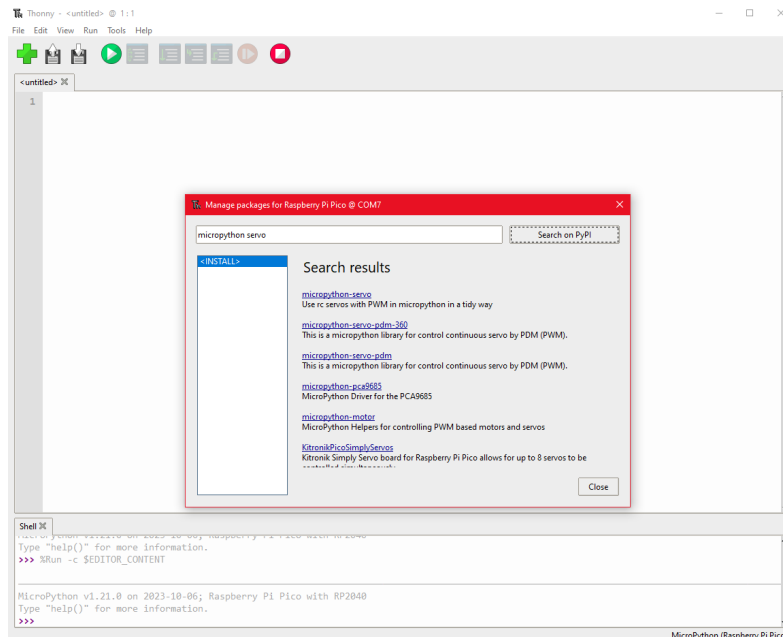
Modulacja sygnału to temat, który wymaga zaawansowanej wiedzy matematycznej i elektronicznej, aby go dogłębnie zgłębić, co wykracza poza zakres tego projektu. Dla ułatwienia dostępna jest biblioteka MicroPython o nazwie servo.py, która wykonuje wszystkie niezbędne funkcje i równania w tle, dzięki czemu sterowanie kątem serwa staje się proste i wystarczy wprowadzić wybrany kąt.

Musisz dodać ten pakiet do swojego Raspberry Pi Pico. Postępuj zgodnie z poniższymi instrukcjami.

Otwórz Thonny Python i utwórz nowy plik, klikając Plik → Nowy lub naciskając Ctrl+N na klawiaturze. Upewnij się, że Pico jest podłączone, a następnie kliknij Narzędzia → Zarządzaj pakietami.



W pasku wyszukiwania wpisz „micropython-servo” i kliknij Szukaj.

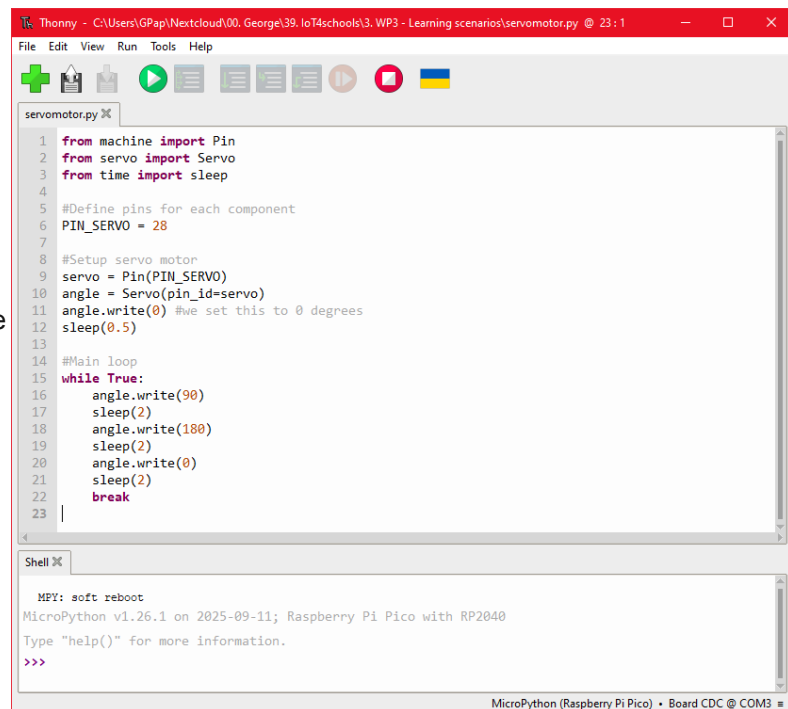


Kliknij *micropython-servo*, a następnie kliknij *Install* (Instaluj). Poczekaj na zakończenie instalacji i kliknij *Close* (Zamknij).

Jesteś gotowy, aby utworzyć program sterujący serwowmotorem.

Ten skrypt MicroPython steruje serwowmechanizmem SG90 w Raspberry Pi Pico W, a następnie przerywa program po powrocie do 0°.

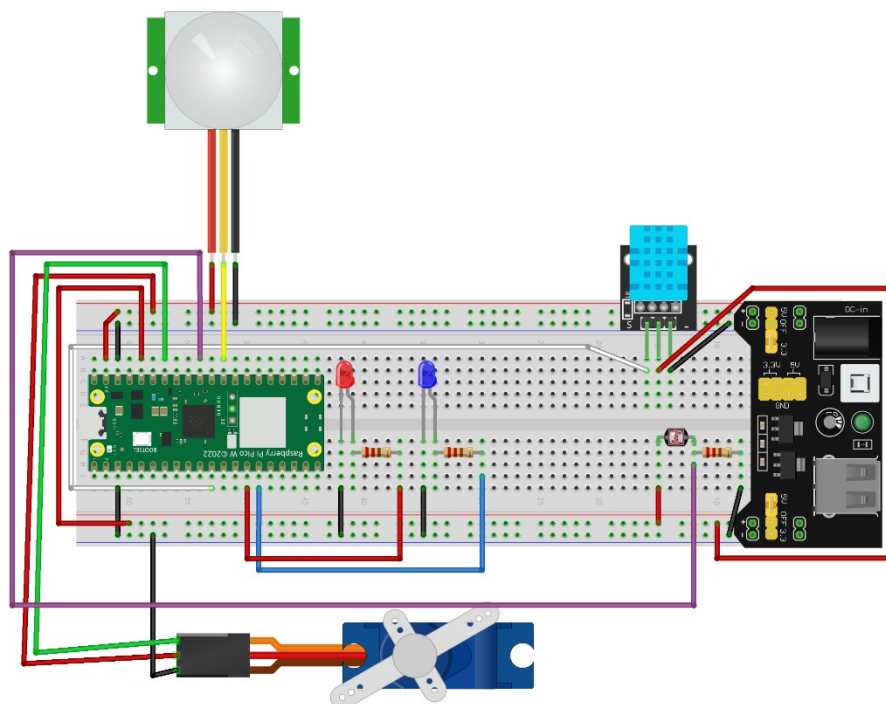
- Konfiguracja: `PIN_SERVO = 28`. Tworzysz pin na GP28 (`servo = Pin(PIN_SERVO)`) i tworzysz kontroler serwowmechanizmu na tym pinie (`angle = Servo(pin_id=servo)`). Centrujesz go w punkcie 0° za pomocą `angle.write(0)` i czekasz 0,5 s.
- Pętla główna: steruje 90° → czekaj 2 s, 180° → czekaj 2 s, następnie 0° → czekaj 2 s.
- Wyjście: `break` natychmiast opuszcza pętlę po osiągnięciu 0°, więc sekwencja wykonuje się raz i zatrzymuje.



2.5 Poziom 2 – Integracja systemu & Adafruit IO

2.5.1 Proces budowy obwodu

Czas na stworzenie kompletnego układu. Postępuj zgodnie ze schematem poniżej, aby połączyć całą elektronikę i stworzyć ekosystem „SmarHome”. Upewnij się, że Pico jest odłączone od komputera PC/laptopa i że podłączyłeś prawidłowe piny GPIO.



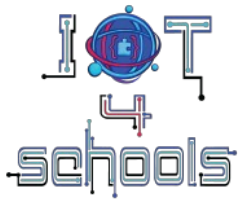
fritzing

2.5.2 Programowanie

Poniższy skrypt w MicroPython-ie to kompilacja wszystkich poprzednich skryptów opracowanych w ramach tego projektu, ułożonych w sposób symulujący dwie różne wersje systemów inteligentnego domu:

Automatyczny system inteligentnego domu, działający w tle i oferujący następujące funkcje:

- Inteligentny system regulacji temperatury, który ma dwie funkcje: 1. Mierzy temperaturę w pomieszczeniu i używa serwomechanizmu do otwarcia „okna”, gdy temperatura wzrośnie powyżej określonego przez użytkownika progu. 2. Mierzy wilgotność w pomieszczeniu i używa serwomechanizmu do zamknięcia „okna”, gdy wilgotność wzrośnie powyżej określonego przez użytkownika progu (na przykład w przypadku ulewnego deszczu na zewnątrz).
- Inteligentny system oświetlenia, który ma jedną funkcję: Wykorzystuje fotorezystor LDR do pomiaru poziomu światła i włącza diody LED, gdy poziom światła spadnie poniżej określonego progu (na przykład w nocy).
- Inteligentny system bezpieczeństwa, który ma jedną funkcję: Wykorzystuje czujnik ruchu PIR do wykrywania ruchu, a po jego wykryciu wydaje polecenie włączenia diod LED i zamknięcia „okna” (serwomechanizmu).



```
from machine import Pin, ADC
from dht import DHT11
from servo import Servo
from time import sleep

#Define pins for each component
PIN_DHT = 8
LED_RED = 10
LED_BLUE = 11
PIN_PIR = 22
PIN_LDR = 26
PIN_SERVO = 28

#Setup DHT11
dht = DHT11(PIN_DHT)

#Setup LED lights
ledred = Pin(LED_RED, Pin.OUT)
ledblue = Pin(LED_BLUE, Pin.OUT)
ledred.value(0)
ledblue.value(0)

#Setup PIR
PIR = Pin(PIN_PIR, Pin.IN)

#Setup LDR
LDR = ADC(Pin(PIN_LDR))

#Setup servo motor
servo = Pin(PIN_SERVO)
angle = Servo(pin_id=servo)
angle.write(0) #we set this to 0 degrees
sleep(0.5)

#Global variables
SERVO_CLOSED = 0
SERVO_OPEN = 90
TEMP_threshold = 28 #adjust as needed
HUM_threshold = 75 #adjust as needed

#Main loop
while True:
    dht.measure()
    temp = dht.temperature()
    hum = dht.humidity()
    print("Room Temperature:", temp, "°C")
    print("Room Humidity:", hum, "%")
    sleep(5)

    #Smart temperature system
    if temp > TEMP_threshold:
        angle.write(SERVO_OPEN) #adjust angle as needed
        sleep(2)
    elif hum > HUM_threshold:
        angle.write(SERVO_CLOSED) #adjust angle as needed
        sleep(2)
    else:
        pass #keep current position

    #Smart lights system
```

```
light_level = LDR.read_u16()
if light_level > 5000: #adjust as needed
    ledred.value(1)
    ledblue.value(1)
    sleep(5)

#Smart security system
if PIR.value() == 1:
    print("Motion Detected")
    ledred.value(1)
    angle.write(SERVO_CLOSED)
    sleep(2)
    break
```

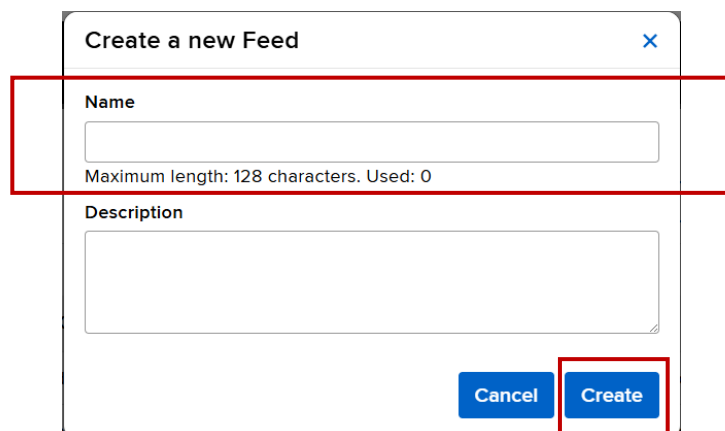
Aplikacja Smart Home, która tworzy aplikację online z wykorzystaniem platformy Adafruit IO, sterowaną przez użytkownika, i posiada następujące funkcje:

- Przełącznik LED włącz/wyłącz
- Serwomechanizm otwierania/zamykania
- Monitorowanie temperatury/wilgotności
- Monitorowanie bezpieczeństwa

Do aplikacji Smart Home zalecamy użycie Adafruit IO. Najpierw należy założyć bezpłatne konto na stronie <https://io.adafruit.com>. Następnie należy określić dane, które mają być wysyłane do chmury. W tym projekcie chcemy wysyłać dane z czujników do aplikacji i móc wysyłać polecenia z aplikacji do systemu Smart Home. W tym celu należy utworzyć pięć „kanałów”. Kanały służą do przechowywania danych. Aby utworzyć kanał, należy przejść do zakładki „Feeds” i wybrać „New Feed”.



Otworzy się okno dialogowe, w którym należy nadać nazwę kanałowi. Dostępny jest również opcjonalny opis.



Utwórz pięć kanałów: temperatura, wilgotność, PIR, serwo, diody LED.

[gpapaioannou](#) / Feeds

[Help](#)

[+ New Feed](#)

[+ New Group](#)

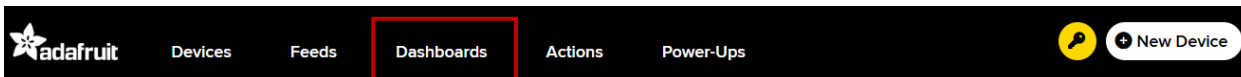
[Q](#)

Default

[+](#) [...](#)

Feed Name	Key	Last value	Recorded
☐ humidity	humidity		1 minute ago
☐ leds	leds		less than a minute ago
☐ pir	pir		less than a minute ago
☐ servo	servo		less than a minute ago
☐ temperature	temperature		1 minute ago

Następnym krokiem jest utworzenie pulpitu nawigacyjnego. Wybierz zakładkę „Dashboards”, a następnie „New dashboard”. Pojawi się okno, w którym należy wpisać nazwę wybranego pulpitu nawigacyjnego.



[gpapaioannou](#) / Dashboards

[Help](#)

[+ New Dashboard](#)

[Q](#)

Name	Key	Created At
------	-----	------------

Create a new Dashboard

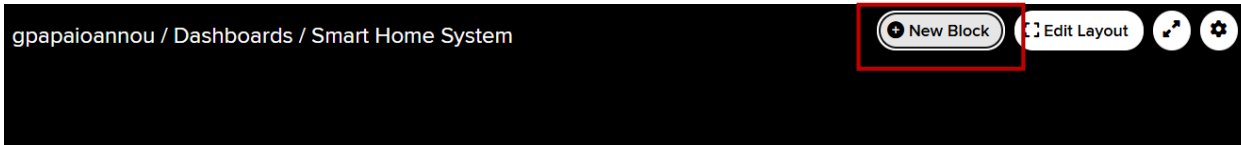
Name

Description

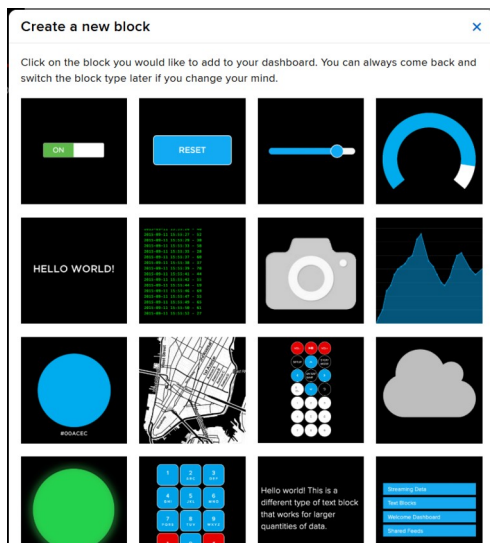
Cancel

Create

Następnie kliknij nazwę swojego pulpitu, aby do niego wejść. Następnie po prawej stronie kliknij „Nowy blok”.



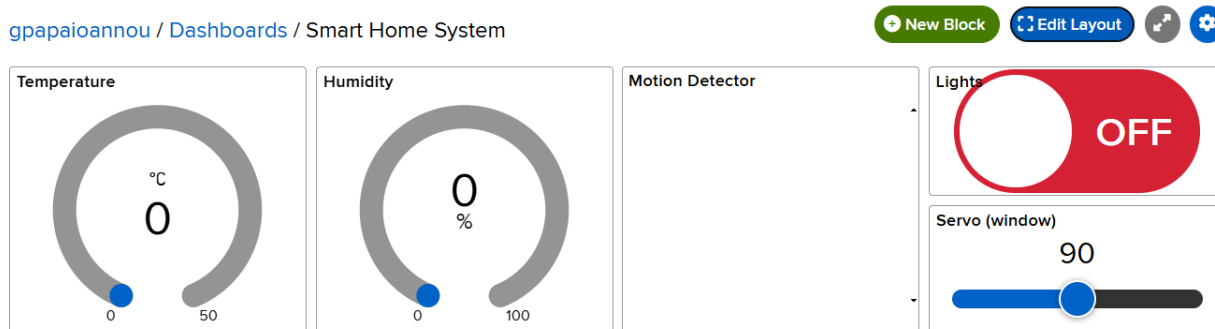
Adafruit IO oferuje różne bloki do wyświetlania danych (pola tekstowe, wskaźniki, wykresy, mapy itp.). W tym projekcie będziemy potrzebować jednego przełącznika, jednego suwaka, dwóch wskaźników i jednego bloku strumienia.



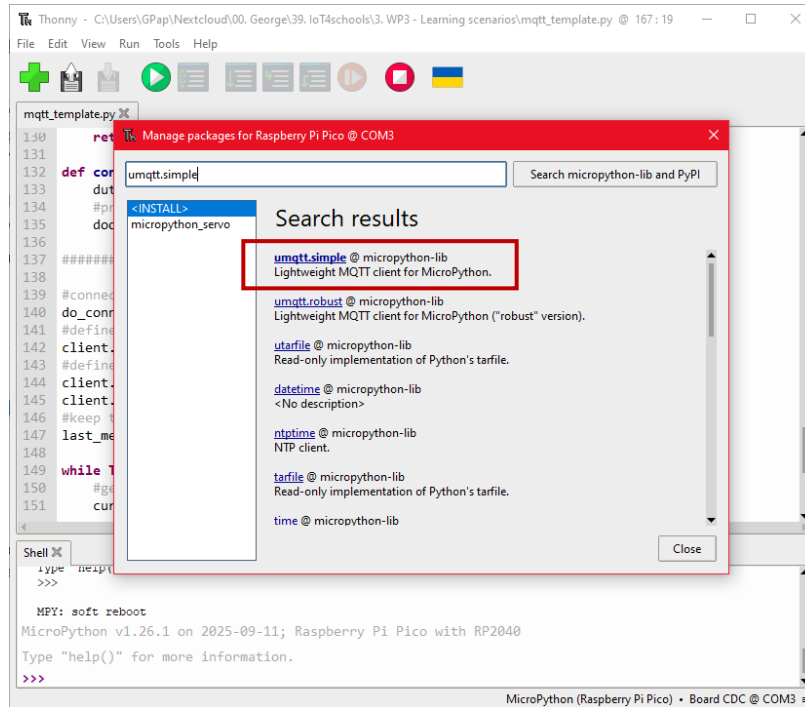
Tworząc nowe bloki, upewnij się, że są one powiązane z odpowiednim kanałem:

- Blok temperatury powinien być powiązany z kanałem „temperature”.
- Blok wilgotności powinien być powiązany z kanałem „humidity”.
- Blok oświetlenia powinien być powiązany z kanałem „leds”.
- Blok serwo powinien być powiązany z kanałem „servo”.
- Blok czujnika ruchu powinien być powiązany z kanałem „pir”.

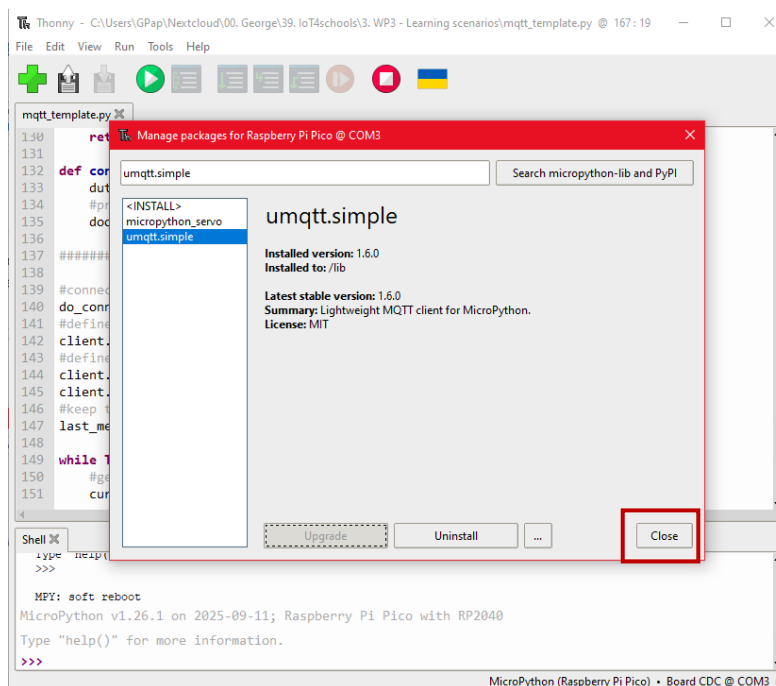
Twój panel powinien wyglądać jak na obrazku po skonfigurowaniu wszystkiego. Jeśli chcesz, możesz edytować układ, klikając przycisk „Edit Layout”.

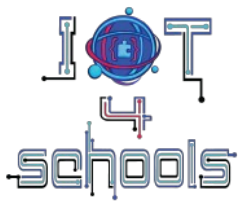


Przejdźmy teraz do edytora Thonny i zainstalujmy bibliotekę *umqtt.simple*. W tym celu wybierz „Narzędzia”, a następnie „Zarządzaj pakietami”. Pojawi się okno, w którym należy wpisać nazwę biblioteki, którą chcesz zainstalować – w tym przypadku *umqtt.simple*. Wybierz bibliotekę i kliknij „Instaluj”.



Wybierz bibliotekę i kliknij „Instaluj”. Po pomyślnym zakończeniu instalacji kliknij „Zamknij”.





Zainstalujmy teraz bibliotekę `iot4schools.py`, którą można pobrać z GitHuba (<https://github.com/angsam/iot4schools>) lub ze strony projektu na stronie IoT4schools. Aby to zrobić, w edytorze Thonny otwórz plik `iot4schools.py`, a następnie kliknij Plik → Zapisz jako, wybierz Raspberry Pi Pico, nazwij go `iot4schools.py` i kliknij „OK”.

Teraz musisz wrócić do swojego kodu i dodać elementy niezbędne do przesyłania danych do aplikacji inteligentnego domu w Adafruit.

Najpierw musisz zaimportować bibliotekę `iot4schools.py`. Aby to zrobić, na początku kodu wpisz następujący wiersz:

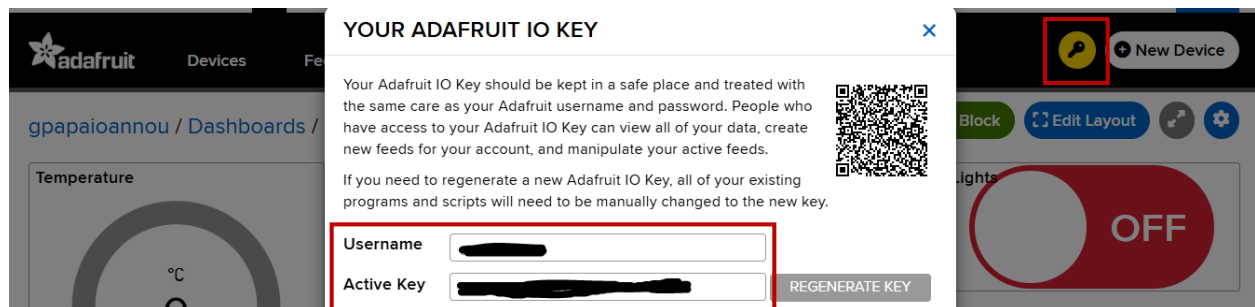
```
import iot4schools
```

Następnie należy skonfigurować połączenie Wi-Fi. W tym celu potrzebny jest następujący wiersz kodu:

```
#Setup WIFI connection
WIFI_SSID = "your_wifi_name"
WIFI_PASSWORD = "your_wifi_password"
ADAFRUIT_IO_USERNAME = "your_username"
ADAFRUIT_IO_KEY = "your_key_from_adafruit"

iot4schools.connect_wifi(WIFI_SSID, WIFI_PASSWORD)
```

ADAFRUIT_IO_USERNAME i ADAFRUIT_IO_KEY znajdują się na Twoim koncie Adafruit IO. Wystarczy przejść do konta Adafruit IO, kliknąć ikonę „klucza”, a zobaczysz swoją nazwę użytkownika i klucz IO.



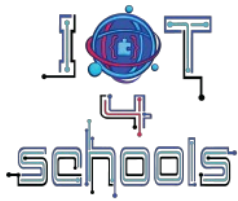
Następnym krokiem jest użycie protokołu MQTT (*Message Queuing Telemetry Transport*) do wysyłania danych do Raspberry Pi Pico, a także do ich odbierania. Aby to zrobić, należy dodać do skryptu następujący kod:

```
#Setup MQTT
client = iot4schools.create_mqtt_client(ADAFRUIT_IO_USERNAME, ADAFRUIT_IO_KEY)
iot4schools.connect_mqtt(client)
```

Teraz możesz zapisać ścieżki do kanałów i utworzyć funkcję, która wysyła dane do kanałów. Aby to zrobić, zacznij od utworzenia zmiennych przechowujących te ścieżki. Musisz powtórzyć ten proces dla wszystkich posiadanych kanałów. Użyj poniższego kodu:

```
#Setup FEEDS
FEED_TEMP = "your_username/feeds/temperature"
FEED_HUM = "your_username/feeds/humidity"
FEED_LEDS = "your_username/feeds/leds"
FEED_PIR = "your_username/feeds/pir"
FEED_SERVO = "your_username/feeds/servo"
```

Aby utworzyć funkcję wysyłającą dane do Adafruit IO, użyj następującego kodu:



```
#Function to send data
```

```
def Send_Data(temperature, humidity, pir):  
    client.publish(FEED_TEMP, bytes(str(temperature), "utf-8"), qos=0)  
    client.publish(FEED_HUM, bytes(str(humidity), "utf-8"), qos=0)  
    client.publish(FEED_PIR, bytes(str(pir), "utf-8"), qos=0)
```

Należy pamiętać, że w funkcji *Send_Data()* opisane są tylko kanały, które muszą wysłać dane (tj. temperatura, wilgotność, pir). W przypadku pozostałych kanałów (tj. diod LED, serwomechanizmów), którymi chcesz sterować z poziomu Adafruit IO, musisz użyć innego fragmentu kodu w skrypcie. Musisz utworzyć nową funkcję, która będzie odbierać dane z Adafruit IO (np. w celu włączenia/wyłączenia diod LED lub obrócenia serwomechanizmu pod określonym kątem) o nazwie *Receive_Data(feed, msg)*. Użyj poniższego kodu:

```
#Function to receive data
```

```
def Receive_Data(feed, msg):  
    if FEED_LEDS in feed:  
        #control LEDs  
        action = str(msg, "utf-8")  
        print("action = {} ".format(action))  
        if action == "ON":  
            ledred.value(1)  
            ledblue.value(1)  
        else:  
            ledred.value(0)  
            ledblue.value(0)  
    if ADAFRUIT_IO_FEEDNAME4 in feed:  
        #control Servo  
        action = str(msg, "utf-8")  
        print("action = {} ".format(action))  
        angle.write(action)
```

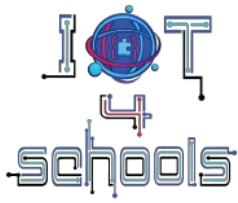
Następnie potrzebujesz kilku dodatkowych linii kodu, aby dane przesłane przez Adafruit IO przełożyły się na działania w Twoim inteligentnym domu. Najpierw musisz wywołać funkcję *set.callback()*, która informuje Raspberry Pi Pico o wysłaniu „wiadomości” za pomocą aplikacji Adafruit IO. Ta linijka kodu wywołuje funkcję *Receive_Data(feed, msg)*, która sprawdza, czy użytkownik chce włączyć/wyłączyć diody LED lub zmienić położenie serwomechanizmu. Następnie, aby program wiedział, które kanały wysłały „wiadomość”, używamy funkcji *subscribe*. Zwróć uwagę, że te funkcje (*set.callback()* i *subscribe*) znajdują się w bibliotece *umqtt.simple*, którą zainstalowałeś w swoim Pico kilka kroków wcześniej.

```
#Define Receive_Data function name to client  
client.set_callback(Receive_Data)
```

```
#Define Feed names to get messages from  
client.subscribe(FEED_LEDS)  
client.subscribe(FEED_SERVO)
```

Na koniec możesz napisać główną pętlę swojego programu:

```
#Main loop  
while True:  
    dht.measure()  
    temp = dht.temperature()  
    hum = dht.humidity()  
    if PIR.value() == 1:  
        print("Motion Detected")  
        break  
  
    Send_Data()
```



```
sleep(30)

#try to read message from client (eg from FEED_SERVO)
try:
    client.check_msg()
except KeyboardInterrupt:
    print("\Ctrl-C pressed...exiting")
    client.disconnect()
    sys.exit()
```

W pętli głównej skrypt prosi czujnik DHT o pomiar temperatury i wilgotności. Prosi czujnik PIR o informację o wykryciu ruchu, a następnie wysyła te dane do Adafruit IO. Program czeka 30 sekund przed ponownym sprawdzeniem.

Następnie skrypt sprawdza, czy odebrano komunikat z aplikacji Adafruit IO. Na koniec, jeśli użytkownik chce zatrzymać działanie skryptu, naciskając Ctrl+C, skrypt rozłącza Pico z aplikacją Adafruit IO i wyłącza system.

Poniżej przedstawiono skompilowany kod łączący Raspberry Pi Pico z aplikacją Adafruit IO:

```
from machine import Pin
from dht import DHT11
from servo import Servo
from time import sleep

import iot4schools

#Setup WIFI connection
WIFI_SSID = "your_wifi_name"
WIFI_PASSWORD = "your_wifi_password"
ADAFRUIT_IO_USERNAME = "your_username"
ADAFRUIT_IO_KEY = "your_key_from_adafruit"

iot4schools.connect_wifi(WIFI_SSID, WIFI_PASSWORD)

#Setup MQTT
client = iot4schools.create_mqtt_client(ADAFRUIT_IO_USERNAME, ADAFRUIT_IO_KEY)
iot4schools.connect_mqtt(client)

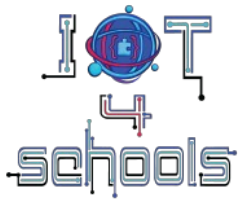
#Define pins for each component
PIN_DHT = 8
LED_RED = 10
LED_BLUE = 11
PIN_PIR = 22
PIN_SERVO = 28

#Setup DHT11
dht = DHT11(PIN_DHT)

#Setup LED lights
ledred = Pin(LED_RED, Pin.OUT)
ledblue = Pin(LED_BLUE, Pin.OUT)
ledred.value(0)
ledblue.value(0)

#Setup PIR
PIR = Pin(PIN_PIR, Pin.IN)

#Setup servo motor
servo = Pin(PIN_SERVO)
angle = Servo(pin_id=servo)
```



```
angle.write(0) #we set this to 0 degrees
sleep(0.5)

#Global variables
SERVO_CLOSED = 0
SERVO_OPEN = 90
TEMP_threshold = 28 #adjust as needed
HUM_threshold = 75 #adjust as needed

#Setup FEEDS
FEED_TEMP = "your_username/feeds/temperature"
FEED_HUM = "your_username/feeds/humidity"
FEED_LEDS = "your_username/feeds/leds"
FEED_PIR = "your_username/feeds/pir"
FEED_SERVO = "your_username/feeds/servo"

#Function to send data
def Send_Data(temperature, humidity, pir):
    client.publish(FEED_TEMP, bytes(str(temperature), "utf-8"), qos=0)
    client.publish(FEED_HUM, bytes(str(humidity), "utf-8"), qos=0)
    client.publish(FEED_PIR, bytes(str(pir), "utf-8"), qos=0)

#Function to receive data
def Receive_Data(feed, msg):
    if FEED_LEDS in feed:
        #control LEDs
        action = str(msg, "utf-8")
        print("action = {}".format(action))
        if action == "ON":
            ledred.value(1)
            ledblue.value(1)
        else:
            ledred.value(0)
            ledblue.value(0)
    if FEED_SERVO in feed:
        #control Servo
        action = str(msg, "utf-8")
        print("action = {}".format(action))
        angle.write(float(action))

#Define Receive_Data function name to client
client.set_callback(Receive_Data)

#Define Feed names to get messages from
client.subscribe(FEED_LEDS)
client.subscribe(FEED_SERVO)

#Main loop
while True:
    dht.measure()
    temp = dht.temperature()
    hum = dht.humidity()
    if PIR.value() == 1:
        print("Motion Detected")
        break

    Send_Data()

    sleep(30)
```

```
#try to read message from client (eg from FEED_SERVO)
try:
    client.check_msg()
except KeyboardInterrupt:
    print("\Ctrl-C pressed...exiting")
    client.disconnect()
    sys.exit()
```

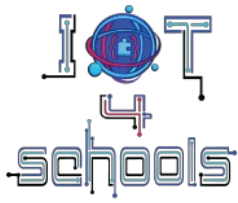
Na koniec, warto zauważyć, że fotorezystor LDR nie został uwzględniony w aplikacji Adafruit IO. Zrobiono to celowo, aby nauczyciele mogli zapoznać się z aplikacją online i kodem niezbędnym do odczytu i wyświetlania danych z czujników.

3 Wskazówki i rekomendacje

1. Praca grupowa 3-4 uczniów sprawdza się najlepiej: przydzielaj rotacyjne role (programista, konstruktor, tester, dokumentalista), aby wszyscy byli zaangażowani i odpowiedzialni. Przygotuj wspólną listę kontrolną dla każdego zespołu na każdą sesję.
2. Zainstaluj MicroPython na komputerach, sprawdź dostęp Wi-Fi dla Pico W (2,4 GHz, WPA2) i wstępnie utwórz kanały Adafruit IO lub strategię kont klasowych przed sesją 6. Zachowaj jeden w pełni sprawny zestaw „wymieny”, aby zminimalizować przestoje.
3. Urządzenia 5 V (np. PIR VCC) powinny być podłączone do szyny 5 V, a urządzenia 3,3 V (diody LED, MCU GPIO) do szyny 3,3 V; zawsze udostępniaj uziemienie (GND). Nigdy nie zasilaj jednocześnie przez USB i baterie.
4. Odczekaj około 30–60 s na stabilizację PIR i około 2 s między odczytami DHT11, aby zachować częstotliwość aktualizacji.
5. W przypadku awarii należy zmieniać po kolei (przewód, linię kodu, komponent) i ponownie testować. Przed uruchomieniem pełnych skryptów należy użyć narzędzia REPL firmy Thonny do sondowania pinów/odczytów czujników.
6. Przed integracją należy przetestować komponenty za pomocą dostarczonych prostych skryptów (mruganie, odczyt DHT11, odczyt PIR/LDR, podstawowe serwomechanizmy).
7. Unikaj „skakania” wokół progów (np. temperatury 28°C), dodając niewielką strefę nieczułości i minimalny czas między ruchami serwomechanizmu. Naucz if/elif/else wymuszania priorytetu (np. nadpisywanie wilgotności), jak pokazano w przykładzie integracji.
8. Uśrednij kilka odczytów ADC z LDR, aby wygładzić szum; udokumentuj, jak wybrano próg (zmiierz „pokój” vs. „ciemność” i ustaw punkt środkowy).
9. Przechowuj nazwę użytkownika/klucz API oddzielnie (stałe na górze pliku) i nie udostępniaj kluczy na zrzutach ekranu.
10. Wymagaj od każdego zespołu jednostronicowego raportu ze schematem okablowania/zdjęciem, kluczowymi fragmentami kodu i zrzutem ekranu pulpitu nawigacyjnego; użyj krótkiej rubryki demonstracyjnej (funkcjonalność, bezpieczeństwo, dokumentacja, praca zespołowa).
11. Odwołaj się do dyskusji na zajęciach na temat zrównoważonego rozwoju (oszczędność energii), prywatności/bezpieczeństwa (ochrona danych, dostęp do chmury z minimalnymi uprawnieniami) oraz korzyści/ryzyka związanego z Internetem Rzeczy (IoT). Powiąż je z konkretnymi wyborami projektowymi podjętymi przez uczniów (np. kiedy publikować dane, co przechowywać).

4 Referencje

1. **Raspberry Pi Pico / Pico W documentation.** Raspberry Pi Ltd. Board overview, GPIO and MicroPython guidance. (<https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>)
2. **Tom’s Hardware – Resistor color codes** (images/decoding examples used in component explanations). (<https://www.tomshardware.com/how-to/resistor-color-codes>)
3. **SparkFun Education – What is a jumper wire?** (Background on jumper types/uses). (<https://blog.sparkfuneducation.com/what-is-jumper-wire>)



4. **IoT4Schools GitHub – `iot4schools.py` helper** (Wi-Fi + MQTT utilities used in Level 2 integration).
(<https://github.com/angsam/iot4schools>)
5. **Lam, J.; Chen, A.; Bennett, A.; Triantafyllou, M. (2023).** *Propeller Characterization Testing of a Blue Robotics T200 Thruster*.
(https://www.researchgate.net/publication/373891538_Propeller_Characterization_Testing_of_a_Blue_Robotics_T200_Thruster)